Cogito, Ergo Feci:* False Success, the Limits of Weight Surgery, and What an External Verifier Catches in a Computer-Use Agent

Anonymous Author(s)
Affiliation
Address
email

Abstract

Computer-use agents often fail by announcing that a task is finished when it is not. We diagnose it in Holo-3.1-4B on OSWorld and test whether editing activations or weights, without training, removes it. The agent scores 56.7% on 119 diagnostic tasks; a false "done" is 57% of its failures, and re-sampled it answers "done" on most false completions too (72%, against 94% on true ones). With a replay that rebuilds each step's input and a gate fixed in advance, we find three traps that would each have produced a positive result: a false-success direction that adds at most 0.04 AUROC to position, run-to-run noise under which only 50% of false successes recur, and a loop direction in a smaller model that beats position yet reduces loops only at a large cost to healthy actions. No steering arm or weight edit passes the gate in pilot screens, which detect only a fall from 8/8 to 4/8 false answers and hold a single feasible false-success context, and no edited or prompted arm run online beats the run-to-run noise of the original. The tool call leans on a thought that already declares success: on 39 feasible contexts, neutral text in its place removes 80–82% of the `answer` margin, yet `answer` stays preferred in 37, perhaps carried by the success claim in the retained reasoning. Of the edits that move the call, a model merge shows no detectable selectivity (+0.7 points, 95% CI −16.1 to +16.3) and a closed-form edit lowers false answers more, with an interval that includes zero. An external verifier, Claude Sonnet (CLI alias, version not pinned) checking each requirement on the final screen, separates false from true successes offline, detecting 68% of false successes and rejecting 10% of true ones (the agent judging itself with a free-form prompt: 53% and 48%); online, on 120 held-out tasks, the guarded agent's false "done" rate fell from 22.5% to 10.8% (−11.7 points, 95% CI −19.2 to −4.2), formally meeting the registered criterion, but about two thirds of the drop come from episodes in which the guard never ran, so we do not attribute it to the guard; the score did not change measurably (+1.6, −4.4 to +7.7).

1 Introduction

Computer-use agents run a closed loop: read a screenshot, act, repeat until they declare the task done or run out of steps. A recurring way to fail is to declare it done when it is not. The agent announces success, the episode ends, and the environment's checker returns zero [31]. Figure 1a shows one such episode: the agent writes that it cannot finish without a password, then calls `answer` instead of

*"I think, therefore I have done", after Descartes: the agent's reasoning concludes the task is done before checking.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

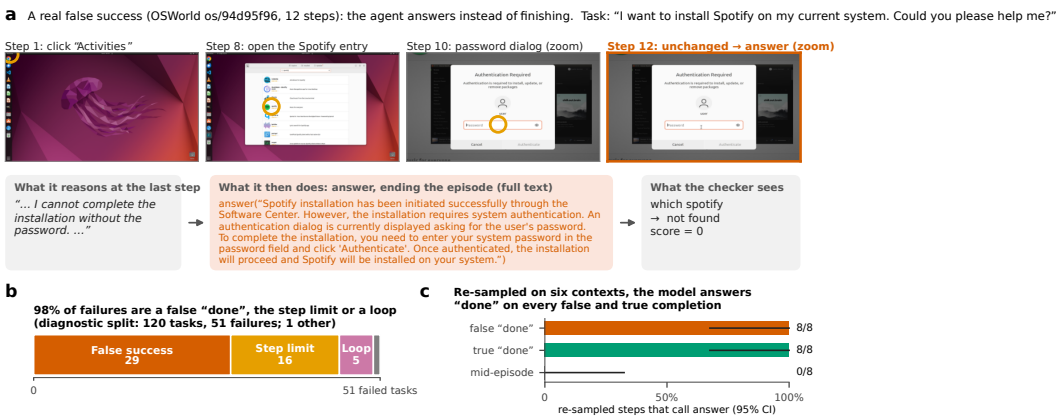


Figure 1: The agent declares success it has not reached. (a) A real episode: the reasoning says "I cannot complete the installation without the password.", and the agent, instead of finishing, calls `answer`, which ends the episode, with a message that the installation "has been initiated successfully" and that the user must enter the password (full text in the figure); the checker scores 0. (b) Failures on the frozen diagnostic split. (c) Re-sampled from the same six contexts ($k = 4$), the original model calls `answer` on 8/8 false-completion and 8/8 true-completion samples, and on 0/8 mid-episode samples; bars show Wilson 95% intervals; one false-completion context is an infeasible task (Appendix M).

finishing, reporting the installation as "initiated successfully" and leaving the password to the user; the episode ends with nothing installed. Users report the same failure in deployment, including an agent that keeps acting while its screen capture fails, then claims completion [6].

Interpretability offers a tempting repair. If a behaviour is mediated by one direction in the residual stream, projecting it out of every matrix that writes into the stream removes the behaviour without a gradient step [2], and "weight-space programming" recently removed a looping habit from a 0.8B model by editing 131,072 weights [37]. False success looks like an ideal target: it is frequent (Fig. 1b) and it is a single discrete decision, the call to the `answer` tool, of the kind single directions are known to steer [4, 43]. This paper is about the gap between finding such a direction and proving that it causes the behaviour, and about what happens when one tries to edit the behaviour away anyway.

Contributions. We report only what we measured.

1. **A diagnosis of a real agent on real desktops** (§2). Holo-3.1-4B solves 66/119 tasks of a frozen OSWorld split; 29 of its 51 failures are false successes, and re-sampled, it still says "done" on most false completions (72%, against 94% on true ones).
2. **Three measured traps and the protocol they impose** (§3): position-confounded directions, regression to the mean between runs, and detection without causation. Each would have produced a false positive here.
3. **A causal record of every intervention we ran** (§4), from steering to mechanism-level weight edits and model merging, as pre-registered pilot screens with a semantic audit of every changed action. None passes the gate, and the screens could find only large effects: the gate flags a fall to 4/8 false answers or fewer, and one of the two false-success screening contexts is an infeasible task, so the evidence on feasible false successes rests on a single context and a few samples per arm.
4. **A mechanistic account of the failure to edit** (§4.1, §4.3): the tool call leans on a thought that already states completion, and of the two edits that move it, a merge shows no detectable selectivity on a strictly feasible cohort, while the selectivity of a closed-form edit is not established, in line with probes that separate the two no better than position. No edited or prompted model arm run online beats the run-to-run difference of the original (§4.4).
5. **An external verifier** (§4.4, Appendix G): offline it detects 68% of false successes at 10% false rejections, where the agent judging itself with a free-form prompt does not separate them; the verifier is Claude Sonnet through an unpinned CLI alias. Online, in one TEST run, the guarded agent's false "done" rate fell from 22.5% to 10.8%, formally meeting its registered criterion, but

about two thirds of the drop come from episodes in which the guard never ran, so we do not attribute it to the guard; the score did not change measurably.

2 Setting and diagnosis

Models and harness. Holo-3.1 models are fine-tunes of the Qwen family [10], released as Qwen3.5 vision-language models with a hybrid decoder: three Gated DeltaNet layers for every full-attention layer, and tied embeddings; the 4B has 32 layers of width 2560, 24 DeltaNet and 8 full attention. At each step the agent writes a JSON object with a `note`, a `thought`, then the `tool_call`, after its reasoning. We reuse the Holo agent loop as packaged by RealtimeGUI [1], a third-party reproduction whose infeasibility rule is broader than the official adapter's (absolute scores are not strictly comparable with H Company's; paired comparisons are), served by vLLM 0.23 [20] with thinking enabled, $T = 0.8$, at most 50 steps and the three latest screenshots. OSWorld [45] tasks run unmodified in Docker+KVM VMs. From 361 tasks we freeze a 120-task DIAG split and a 120-task TEST split that no construction or screening read; 5 online runs and the guarded agent used it (§4.4; Appendix B).

Most failures are a false "done". Each episode gets one category from rules over its actions, screenshots and score, hashed before the official direction analysis: a *false success* calls `answer` on a feasible task scored 0 or claims completion on an infeasible one; FS+ marks such a final `answer` step and FS− that of a true completion. Holo-3.1-4B scores 56.7% (official OSWorld score, partial credit included, the convention for every score here) and fully solves 66 of 119 tasks (55.5%, 95% CI 46.5–64.1). Of 51 failures, 29 (57%) are false successes, 16 have the step limit as primary category and 5 loop. False successes occur in every domain but one and are not early quits (median 18 steps; Appendix C).

The model says "done" on most false completions too. Re-sampled on six screening contexts ($k = 4$, production sampling), the original model calls `answer` on 8/8 false-completion and 8/8 true-completion samples, and on 0/8 mid-episode ones (Fig. 1c); on the strictly feasible cohort of §4, it answers on 55/76 (72%) false and 75/80 (94%) true completions. 9 of the false successes are infeasible tasks on which the model claimed completion; there, `answer` with a refusal is the correct tool, so these contexts are flagged wherever $P(\texttt{answer})$ is counted (Appendix M); yet 8 of 10 final answers on infeasible tasks claim success, so they are mostly false successes, not misworded refusals. Every re-sampled reasoning trace at a completion step opens with a success claim at generated token 4–5, with no verification word before it, on true completions as on false ones (0/16). Any repair must change a decision taken in most cases of both kinds, in only one of them.

3 Three traps, and the protocol they impose

We never record activations during rollouts. Each step's exact chat is rebuilt and replayed in `transformers`; the rebuilt messages match those sent on 7/7 steps, the prefill length matches vLLM's on 12/12, and the replayed model gives its own actions a mean log-probability of -0.16 per token against -1.10 for synthetic ones (Appendix D).

Trap 1: the direction measures position. False completions come later in an episode than true ones. A first FS+/FS− difference-of-means direction reached AUROC 0.836, against 0.870 for the step index alone. The pre-registered analysis (41 FS+ and 109 FS− steps, 95 tasks, task-grouped cross-validation) finds a best direction at 0.76 against 0.724 and 0.735 for the step and context-length baselines; a logistic model on position alone scores 0.722, and adding the best direction raises it by at most 0.04 (Fig. 2b, Table 1). We find no false-success signal beyond the length of the episode.

Trap 2: regression to the mean. Re-running the unmodified agent on the same tasks, only 9/18 (50%, 95% CI 29–71) pass-1 false successes recur (Fig. 2c). A patch evaluated on "false-success tasks before and after" would be credited with about 50 points of reduction while doing nothing.

Trap 3: detecting is not causing. On Holo-3.1-0.8B in MiniWoB++, a loop direction reaches task-grouped AUROC 0.893, far above 0.70 for the step index (Fig. 2a), and is a poor lever: erasing

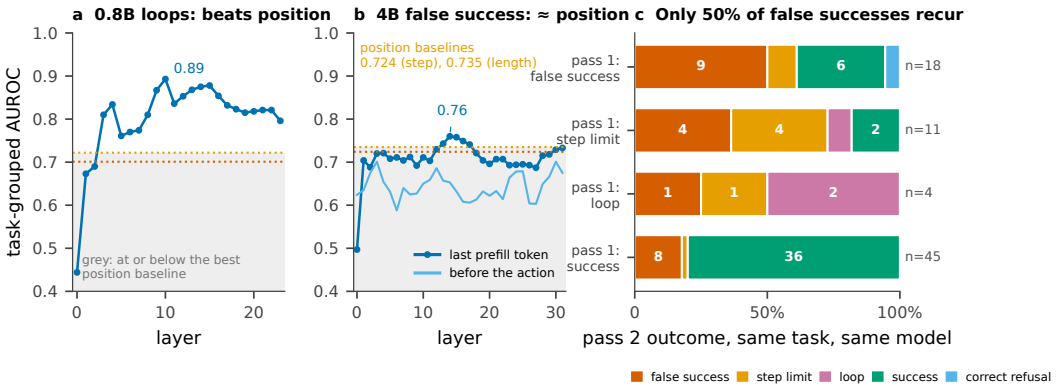


Figure 2: Two of the three traps. (a) Holo-3.1-0.8B, MiniWoB++: a loop direction separates loop steps far above the step-index (red) and context-length (orange) baselines, with task-grouped cross-validation. It still fails every causal test (§H). (b) Holo-3.1-4B, OSWorld, official analysis on both passes (41 FS+, 109 FS−, 95 tasks): the false-versus-true completion direction stays within noise of the position baselines at its best layer (0.76 against 0.724 and 0.735, standard error 0.05), with or without position controlled (Table 1). The grey band marks AUROCs at or below the best baseline. (c) The unmodified 4B run twice on the same tasks (80 tasks scored in both passes). Only 9 of 18 pass-1 false successes recur, against 36 of 45 pass-1 successes.

Predictor (task-grouped 5-fold CV)	Best layer	AUROC
Step index alone	–	0.724
Context length alone	–	0.735
Direction, last prefill token	14	0.760
Direction, before the action	3	0.701
Position (step, log length), logistic	–	0.722
+ direction, last prefill token	15†	0.762
+ direction, before the action	22†	0.717

Table 1: Official false-success direction analysis: no signal beyond position. Every direction AUROC is within noise of the position baselines (standard error about 0.05). Holo-3.1-4B, 41 FS+ and 109 FS− steps from 95 tasks, two passes, 5-fold cross-validation grouped by task, labels frozen before analysis. Exploratory: the second pass stopped at 80 tasks. The 95 tasks include 20 of the 45 confirmation-reserve tasks. †Layer chosen after the fact.

it from every writer leaves loop actions as likely as before, its best weight edit is less selective than a random direction, and steering along it reduces loops only at a large cost to healthy actions (§5).

Protocol. (i) Free generation is the only efficacy measure: each candidate regenerates the full reasoning ($k = 4$, seeds shared across arms), and invalid outputs stay in every denominator. (ii) A gate fixed before any data: on FS+ contexts of tasks not used to build the intervention, $P(\texttt{answer})$ must fall by at least 25 points and 50% relative, while the FS− answer rate and valid healthy actions may fall by at most 5 points; the FS− criterion separates "less false success" from "less success". (iii) Random directions of matched norm and scope, identity hooks and a budget of generation batches fixed before execution. (iv) A confirmation reserve excluded from every construction and screening, kept to confirm a committed candidate; since none was admitted, five non-admitted arms were run on it instead (§4.4), a departure we list as a limitation. Every changed action is read by a blind semantic audit, because a moved tool call is not a correction (§4.3).

4 No selective effect found

4.1 Where the decision is carried

Patching the residual stream from the same task's previous step (a click) at each layer and four positions, the decision is invisible at the end of the prefill and appears only at the tool-name position from about layer 16 upward (Fig. 3). Three attention heads, which read the system-prompt definition

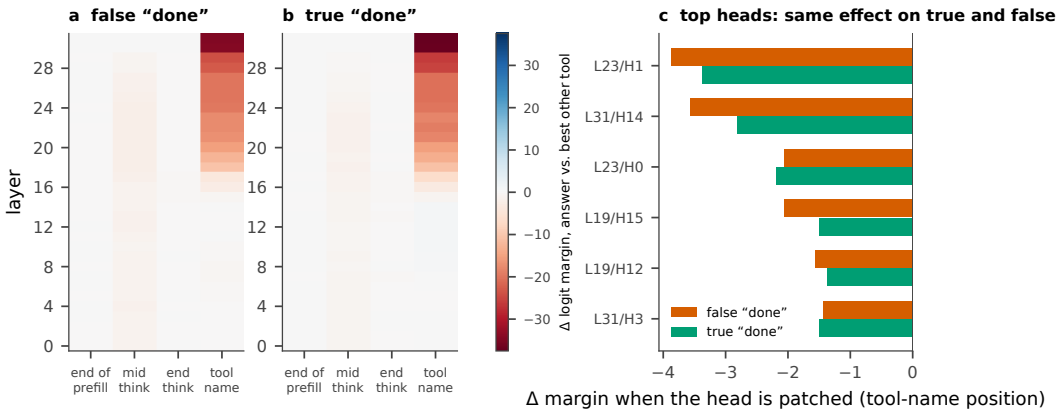


Figure 3: The tool choice becomes visible late, at the tool name, by the same components for true and false completions. Activation patching on Holo-3.1-4B from the previous step of the same task (two tasks per class; exploratory construction data). (a, b) Mean change of the `answer`-versus-best-other-tool logit margin when the residual stream is patched at one layer and position; red lowers `answer`. (c) The six heads with the largest effect at the tool-name position; red bars are false completions, green bars true ones.

of `answer` and the JSON being written, and ten late MLP neurons move the margin, the same way on true and false completions; patching all of them from the donor step lowers it by 11.2–12.2 logits and flips 0 of 4 decisions. At the tool-name position the margin of `answer` over the best other tool is 13.9–19.1 logits on false completions and 10.9–17.8 on true ones. That position follows the JSON `thought` field, which already states that the task is complete, so the margin measures confidence in carrying forward a verbalised decision rather than the margin of the decision.

A ceiling test asks how much must be replaced at once (Fig. 4). With the recorded input forced, whole blocks of writer outputs are swapped in at the decision position from a donor step where the model clicked. On the first FS+ context (margin +16.9), all 32 MLPs together bring the margin to +0.9 without flipping it, while the MLPs and token mixers of layers 12–23 flip it (−3.5), as do those of layers 24–31 (−10.0); a second FS+ context (−3.0) and a true completion (−2.2) flip the same way. Self-swap and residual swap reproduce the baseline and the donor exactly. The first context is an infeasible task, where `answer` is correct; the donor differs in screen, thought and position, not only in decision; and since the true completion flips too, the test locates where the next tool token is assembled, not a mechanism of false success. Removing one direction of the swap difference from the writers of layers 12–23 brings both FS+ margins near zero (+1.4, +1.6) but also two true-completion margins (+4.8, +2.8). On the 19 false and 20 true completions of a strictly feasible cohort, teacher-forced with the screen, history and reasoning fixed, replacing the note and thought by a neutral text of the same token length removes 14.0 logits of margin on false completions (95% CI 12.9–15.1) and 14.5 on true ones (12.9–16.0), 82% and 80% of it (difference −0.46, −2.36 to +1.50), yet `answer` stays preferred in 37 of 39 contexts: the note reinforces the conclusion; the success claim left in the retained reasoning may carry the rest. A counterfactual “not complete yet” text nearly cancels the margin on average, and no probe beats the step index after correction (best 0.64 against 0.70; detecting AUROC below 0.725–0.755 was underpowered). A first two-context version with a donor’s text flips the decision (Appendix F.2). Blanking the inner tokens of the note and thought with spaces at fixed positions, a different neutralisation, lowers the margin by 7.55 and 8.18 logits, with no detectable difference between strata (−0.64, 95% CI −2.13 to +0.80); this smaller effect is partly carried by single DeltaNet layer outputs (layers 18, 22, 28 and 30, 0.9–1.6 logits each, not additive), while blocking any single head’s final-query attention costs at most 0.12 (exploratory, same cohort).

4.2 Steering and targeted edits

Table 2 lists every intervention on the six screening contexts, each testing a hypothesis from the literature: tool choice is linearly steerable [43]; the decision is taken during reasoning; false success is excess expected value or commitment [16, 18]; late MLP components write it [7], tested here at the neuron level; the model fails to check; false success is a perception of completion; the model opens

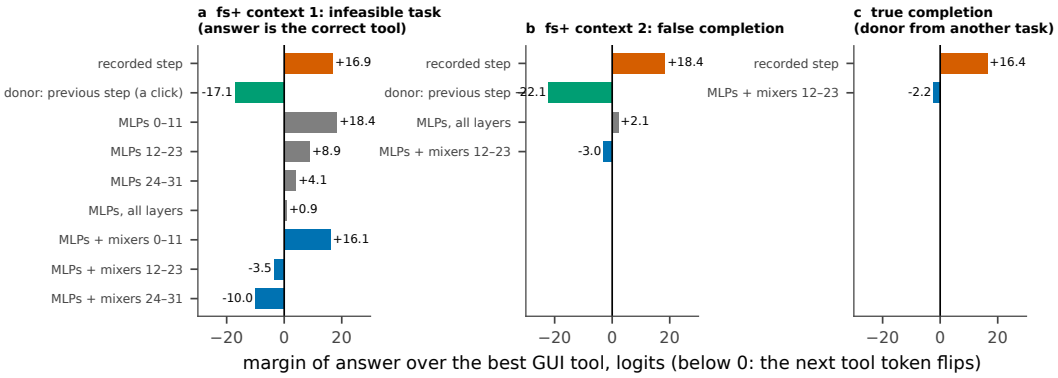


Figure 4: Ceiling test: the next tool token flips only when MLPs and token mixers of a whole block are swapped, on false and true completions alike. Holo-3.1-4B, recorded input forced, logits only. Bars give the margin of `answer` over the best GUI tool after swapping in, at the decision position, the outputs of the indicated writers from a donor step where the model clicked. (a) The first FS+ context is an infeasible task, where `answer` is the correct tool. (c) The true completion uses a donor from another task. Three contexts; exploratory.

with its conclusion; memory of the early episode fades; the circuit of §4.1 carries it. No arm passes the gate. Removing the whole circuit answers on every false completion, like its random control. Pushing toward low value makes an LLM-agent reader find an explicit intention to re-evaluate in 3/8 FS+ traces against 1/8 at baseline, with 0 verification actions and the same answer rate: the model writes that it should check, then answers. A 2560-weight edit of the output row of "Wait" lowers false answers to 6/8 with every control intact, short of the 50% criterion, and an audit rates none of its replacement actions useful. A conditional variant yields *intentions* to correct (the model states the right dates, then generates an unexecuted click on a summary that no evidence shows to be editable) and transfers to 0/12 useful actions on held-out tasks.

Each efficacy cell holds 8 generations from two tasks; 8/8 has a Wilson interval of [68, 100]%, so these screens exclude only large effects. The doses tested, 0.13 to 2 times the residual norm, may be too small to matter, and a ceiling effect is likely.

4.3 Weight edits: levers without established selectivity

Weight edits that need no direction or that target a mechanism (Appendix N) were served on separate hardware against a re-served original, with the same 8 seeds on 6 contexts. Extrapolating the fine-tune away from Qwen, $W(\lambda) = W_{\text{Qwen}} + \lambda(W_{\text{Holo}} - W_{\text{Qwen}})$, called ExPO [52], leaves false answers at 16/16 in all arms but one (15/16). The mechanism-level edits H2–H10 (memory gates, MLPs or all language tensors moved toward Qwen, the end-of-thinking embedding, the attention temperature, the rotary share of the query gain) leave the tool call in place, break generation, or move it only on the infeasible context, where theme clicks replace an answer that should have explained the impossibility; a blind audit finds no useful action (Table 8). Two-neuron conditional circuits, an adaptation of the weight-space programming recipe and an explicit instruction to check each requirement, injected into the reasoning, do no better. Two edits move the decision on feasible contexts. Merging a fraction β of the task vector of CUA-Lite, a Qwen3.5-4B fine-tune on desktop data, lowers false-completion answers to 15/16 and 11/16 at $\beta = 0.25$ and 0.5, but true-completion answers further, to 11/16 and 2/16 (on the feasible FS+ context alone, 3/8 at $\beta = 0.5$): the model continues instead of concluding, whatever the truth of the completion. Online, the merge shows no detectable gain over an original also served on Modal: +1.9 points over 165 tasks (95% CI −4.3 to +8.0; §4.4).

A closed-form edit. A closed-form edit of the 24 residual writers of layers 12–23, in the spirit of AlphaEdit [9] (a ridge update toward the donor displacement, projected off protected keys), moves the decision too: on contexts outside its construction, false-completion answers fall from 15/16 to 8/16 (all on one of two contexts) and true-completion answers from 15/16 to 3/16, with healthy actions intact (on a finished e-mail task, three replacements would write into an already correct field), while a random update of matched per-writer norm leaves 16/16 and 16/16: a real causal lever on stopping, unselective on these contexts. On a strictly feasible cohort of 19 false and 20 true

Hypothesis & intervention	Where	Dose	Out.	fs+ `answer` [95% CI] ↓	fs− `answer` =	healthy valid =	invalid	Verdict
Original (reference)	–	–	–	8/8 [68, 100]	8/8	8/8	0/24	–
`4b-x`: residual addition, matched fs direction	L16, last prefill, $\alpha = -1.20$	0.13	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`4b-x`: residual addition, matched fs direction	L24, last prefill, $\alpha = -5.54$	0.23	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`think`: addition of the "conclusion" direction	L31, every reasoning token, $\alpha = -0.5$	0.25	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low value (Value Axis)	L24, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low commitment (Value Axis)	L24, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low value (Value Axis)	L18, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low commitment (Value Axis)	L18, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`neurons`: zero the top-16 MLP neurons writing `answer`	L20–31, whole generation	–	4	4/4 [51, 100]	–	–	0/4	rejected (bound)
`neurons`: zero the top-64 MLP neurons writing `answer`	L20–31, whole generation	–	4	4/4 [51, 100]	–	–	0/4	rejected (bound)
`circuit`: scale C1 (3 heads + 10 neurons) by $1 - \alpha$	L19/23/31 heads, L30–31 neurons, $\alpha = 0.5$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`circuit`: scale C1 (3 heads + 10 neurons) by $1 - \alpha$	L19/23/31 heads, L30–31 neurons, $\alpha = 1.0$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`circuit`: random control: same-norm Gaussian direction removed	L19/23/31 heads, L30–31 neurons, $\alpha = 0.5$	–	24	8/8 [68, 100]	8/8	8/8	0/24	control
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 0.5$	0.09	24	5/8 [31, 86]	8/8	7/8	0/24	rejected
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 1$	0.17	24	4/8 [22, 78]	8/8	5/8	0/24	rejected
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 2$	0.34	24	5/8 [31, 86]	8/8	7/8	0/24	rejected
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 4$	0.68	24	5/8 [31, 86]	8/8	5/8	0/24	rejected
`perceive`: random control, same norm	L15, last prefill, $\alpha = 4$	0.68	24	7/8 [53, 98]	8/8	6/8	0/24	control
`verify`: add a "verify" vs. "conclude" direction (CAA)	L18, prefill + 20 first tokens, dose 1	1.00	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`verify`: add a "verify" vs. "conclude" direction (CAA)	L18, prefill + 20 first tokens, dose 2	2.00	22	8/8 [68, 100]	8/8	6/6	0/22	rejected (bound)
`verify`: random control, same norm	L18, prefill + 20 first tokens, dose 1	1.00	24	8/8 [68, 100]	8/8	8/8	0/24	control
`verify`: random control, same norm	L18, prefill + 20 first tokens, dose 2	2.00	20	8/8 [68, 100]	7/8	4/4	0/20	control
`mit1`: raise the "Wait" output row for completion openings (weight edit)	`lm_head`, 1 row, 2,560 weights	–	24	6/8 [41, 93]	8/8	8/8	0/24	rejected
`mit1`: shorten memory of late linear-attention layers (weight edit)	`A_log`, 9 layers, retention $\times 0.8$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`mit1`: lengthen memory of late linear-attention layers (weight edit)	`A_log`, 9 layers, retention $\times 1.25$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected

Table 2: Every causal test on the 4B false-success target. Of the two FS+ contexts of most arms, one is an infeasible task where `answer` is correct, so feasible false successes are represented by a single context and 4 samples per arm; both FS+ contexts of the `perceive` arms are infeasible (Appendix Table 6 gives every count without them). Free generation, $k = 4$, seeds shared across arms, six screening contexts from five tasks (two per stratum). "fs+ answer" must fall (gate: -25 points and -50% relative); "fs− answer" and "healthy valid" must stay within 5 points. "Out." is the number of outputs generated out of 24 planned. "Dose": perturbation norm over the mean residual norm at that layer (nominal; – for neuron and head scalings). Brackets: Wilson 95% interval of the fs+ answer rate, ignoring clustering by task. Every screen is a pilot that can only detect large effects, a fall to 4/8 FS+ answers or fewer. The neuron arms use a different set of six contexts (their own original: 6/8 FS+ answers). *Rejected (bound)*: rejected before completion because the gate had become unreachable. *Control*: random direction, not a candidate. The `perceive` arms use their own screening contexts (reference FS+ 6/8, healthy 6/8). The `verify` dose-2 arm was stopped after 23 outputs when a healthy context looped to the token cap; its counts are from the last snapshot before that output. A `value` arm at L28 was preempted after its 8 FS+ outputs (8/8 `answer`) and has no row; the conditional early-exit variant of the "Wait" edit (5101 weights, §F.3) has no row either.

completions (Appendix F.5), the merge lowers both with no detectable difference (selectivity +0.7, 95% CI −16.1 to +16.3), while the closed-form edit changes false answers by −17.1 points (−28.9 to −6.6) and true ones by −8.8, a selectivity of −8.4 (−22.6 to +5.9) that is not established either way; its single random control itself changes false answers (+15.8 points), so the comparison with the original is the one we report. An edit at or after layer ℓ can treat two states differently only to the extent that some function of the layer-ℓ residual stream separates them, and usefully only if that function generalises to unseen tasks. On the inputs of the 24 writers of layers 12–23, a task-grouped probe separates 19 false completions from 19 true completions no better than position: median AUROC 0.704 at the prefill and 0.634 at the end of reasoning against 0.852 for the same probe on the step index (within-fold, pair-weighted over 71 pairs), or 0.596 and 0.496 against 0.644 pooled out of fold; and 83–86% of the false-completion key energy lies in the true-completion subspace. But 5 of these 19 false completions are infeasible tasks, where `answer` is correct, so the probe mixes in cases with no wrong decision, which weakens the argument; the strictly feasible cohort above avoids this for behaviour, though not for the probe. This is an empirical, non-total overlap with a strong confound of position and length, not an impossibility: a low AUROC for a linear probe

is not an information bound, and even identical subspaces would not force a selective edit to act globally, which would require proportional covariances. The agent's text carries no more: the best text detector reaches AUROC 0.66 against 0.72 for the step index, and chance within the 18 tasks that end both ways; only the hesitation between samples carries a little (AUROC 0.66).

Compiling an edit. Directional ablation at every layer and position equals orthogonalising every writer, the embedding and the vision-merger bias (Proposition 1, Appendix O); a scaled projection compiles exactly, whereas an additive steering vector compiles only as a bias, which Holo-3.1's projections lack. The serving path for a compiled checkpoint is tested on null patches, but no candidate reached it.

4.4 Online evaluation

Five arms were nonetheless run online, each paired by task with an original on the same harness, GPU type and image (Table 3). The three reserve runs of the original, spread over 4.4 points, measure the run-to-run noise: the original against itself differs by -2.3 points across harnesses and +4.4 across servers, comparable to every effect, each within a 95% CI that includes zero: ExPO $\lambda = 1.25$ -2.3, the `answer` output-row edit 3/14 against 5/14, the corrective prompt 5/20 against 6/20, and, against the Modal-served original, the CUA-Lite merge +6.6 (CI -6.7 to +22.1) and the closed-form edit +2.2. A gain will be claimed only if the merge, against an original served on Modal (same GPU type, image and vLLM version, separate account) over the 165 reserve and TEST tasks, has a paired 95% CI above zero, a rule fixed after the merge's reserve scores had been read, so prospective on TEST only: it scores 92.5/165 against 89.3/165 (Δ +1.9, CI -4.3 to +8.0), so by that rule (lower 95% bound above zero) no detectable gain, while two servings of the original differ by +1.5 points on TEST. ExPO then went to the TEST split without passing the gate, a decision we report as such: 70.3/120 against 72.1/120 (Δ -1.5, CI -8.9 to +5.7). On the reserve, paired effects below about 17 points go undetected with 80% power.

An external verifier online. Feci Guard (Appendix G) shows the final screen to a frozen Claude Sonnet verifier, which refuses the answer and names what is missing when a requirement is visibly unmet (at most two refusals). Run once on the 120 TEST tasks under a rule committed in advance, paired with the original on the same endpoint, the guarded agent's false "done" rate fell from 22.5% to 10.8% (-11.7 points, 95% CI -19.2 to -4.2), formally meeting the registered criterion, with no measurable score change (+1.6, -4.4 to +7.7). The fall is not attributable to the guard. In 28 guarded episodes the agent never called `answer`, so the guard never ran; the original had 9 false "done" on these tasks, 9 of the net drop of 14, and never answering occurs about as often in the original's own runs (23 and 20 tasks; McNemar $p = 0.33$ against the guarded arm), as expected from an unseeded agent sampled at $T = 0.8$. On the 92 tasks where the guard could act, false "done" goes from 18 to 13, not tested separately; on the 17 tasks it refused, from 6 to 3. A seeded or neutral-verifier replication has not been run (Table 5). We claim no gain from the guard online.

Imminent completion, not false success. A direction learned inside successful episodes, final `answer` step versus the step before, beats position (AUROC 0.910 at layer 15 against 0.552) but rises before false and true completions alike (14/14 and 14/14 matched pairs), and subtracting it passes no dose of the gate (Appendix F.4).

5 A first negative result: loops in Holo-3.1-0.8B

On MiniWoB++ the 0.8B succeeds on 40.7% of 172 rollouts and loops on 24.5% of failures. Of 40 weight edits along the loop direction, the frozen screening winner (1,953,863 weights) lowers the log-probability of loop actions by 2.31 and that of healthy actions by 8.92 mnat/token, a ratio of 0.26 against 0.31 for a random direction. In free generation, the best steering arm reaches a repeat rate of 81.2% only at a healthy-action rate of 58.3%, and the random direction cannot be told apart from the identity hook (Appendix H). The loop direction is an unselective lever, not a usable one.

Comparison	tasks	first %	second %	Δ points [95% CI]	better/worse
Reserve, noise reference: the original against itself					
Original, three runs, harness/server (local/H200; GitHub/H200; GitHub/Modal)	44–45	43.2 / 46.7 / 42.2		spread 4.4 points	–
Original, local vs. GitHub harness (both served by our H200)	44	43.2	45.5	−2.3 [−13.6, +9.1]	3/4
Original, H200 vs. Modal (both on the GitHub harness)	45	46.7	42.2	+4.4 [−6.7, +17.8]	5/3
Reserve, arms served by our H200, vs. the H200 original on the local harness					
ExPO $\lambda = 1.25$	44	40.9	43.2	−2.3 [−13.6, +9.1]	3/4
`answer` output-row edit (stopped early)	14	21.4	35.7	−14.3 [−50.0, +21.4]	2/4
Corrective prompt (stopped early)	20	25.0	30.0	−5.0 [−20.0, +10.0]	1/2
Reserve, arms served by Modal, vs. the Modal original on the GitHub harness					
CUA-Lite merge $\beta = 0.5$	45	48.9	42.2	+6.6 [−6.7, +22.1]	7/4
Closed-form edit of 24 writers (100%)	45	44.4	42.2	+2.2 [−13.3, +17.8]	7/6
TEST *split (GitHub harness), vs. the H200 original*					
ExPO $\lambda = 1.25$ (had not passed the gate)	120	58.6	60.1	−1.5 [−8.9, +5.7]	11/12
Corrective prompt (partial run)	38	65.8	60.4	+5.3 [−5.1, +15.9]	4/1
Original, H200 vs. Modal (noise reference)	120	60.1	58.6	+1.5 [−4.5, +7.6]	9/8
CUA-Lite merge $\beta = 0.5$, vs. the Modal original	120	58.8	58.6	+0.2 [−6.5, +7.5]	11/10
Primary verdict, rule fixed after the merge's reserve scores were seen: reserve and TEST*, Modal against Modal*					
CUA-Lite merge $\beta = 0.5$	165	56.1	54.1	+1.9 [−4.3, +8.0]	18/14

Table 3: No arm beats the run-to-run difference of the original. OSWorld, paired by task; scores are mean task scores in %, first run against second. Reserve: the 45-task confirmation reserve, never used for construction; each arm is compared with an original on the same harness and the same server, and the first row gives the three reserve runs of the original, the run-to-run noise, and the next two compare the original with itself across harnesses and across servers. The local/H200 run lacks one reserve task (`thunderbird/dd84e895`, scored 1 by the GitHub/H200 run), so the local-versus-GitHub row pairs 44 tasks, on which the GitHub/H200 original scores 20/44 rather than 21/45 on all of them. TEST: ExPO went there without passing the gate; the prompt run is partial. The last row is the primary comparison, with a decision rule fixed before the Modal-original comparison was read but after the merge's reserve scores had been seen (only its TEST part is strictly prospective): a gain only if the lower bound of the CI is above zero. "H200" is our own server, "Modal" a hosted H100 on a separate account per model, both with the same image and vLLM version. 95% CIs by task bootstrap; better/worse counts tasks where the first run scores higher/lower.

6 Discussion and limitations

Why detecting is not causing here. A direction that encodes "this episode is stuck" or "this episode is ending" can be read reliably while the action is computed from features that do not depend on it; probes in computer-use agents can reach high AUC without detecting the intended property [22], and format-confounded probes can lack a functional role [32]. On the 4B, every piece of evidence points to a belief that the task is done, acted on without checking: the model reads the right date on screen and calls the wrong one "next month", or derives the right month, notes that the dates must change, and drops that requirement because the main one is met. If the same belief closes true completions, removing it would suppress them too; the merge, which lowers true completions more than false ones, behaves that way, and the FS− criterion exists to catch it. The ceiling test suggests why narrow edits fail: the next tool token is assembled by many writers at once, after a thought that has already concluded, and the largest doses we tried damaged generation without moving it.

Implications for agent developers. We found no direction whose removal lowers false "done" selectively, in screens able to detect only large effects; within our probes, nothing separated false from true completions beyond the episode's length except the hesitation between samples, and no edit that moved the decision showed an established selectivity on feasible tasks. Our results suggest that a fix must bring in information that we could not find in its activations or text when it stops: an external verifier that checks each requirement of the task against the screen before `answer` is accepted (§4.4, Appendix G), or training that targets false successes, for instance with rewards that penalise episodes ending in a false "done". Weight surgery of the kind tested here has not shown that it can supply it.

Limitations. Related work is discussed in Appendix J. Causal screens use six contexts from five tasks, one of the two FS+ contexts being infeasible, and exclude only large effects; we report "no admissible effect found", not "no effect exists". Online comparisons exclude only effects above about 17 points. The TEST split was used by 5 earlier online runs, one of them ExPO, which had not passed the gate, and then by the guarded agent, and the confirmation reserve, kept for a committed candidate, by five non-admitted arms. Comparisons are many and uncorrected; no intervention on the model is claimed to be a selective repair, and corrected p-values were not computed. The

guarded run's lower false "done" rate meets the primary criterion of its own registration in a single TEST run, but most of it comes from episodes in which the guard never ran, and the deciding control (a seeded or neutral-verifier replication) was not run; the escalation to Holo4-27B is measured in replay only, on 9 episodes. The direction analysis is exploratory because its second pass stopped early. Labels are automatic, with thresholds fixed before analysis but not validated by humans. The harness is a third-party reproduction with a broader infeasibility rule than the official adapter, so absolute scores are not comparable to published numbers. The merge donor's training data may overlap OSWorld, and the Claude verifier may have seen public OSWorld tasks. The primary online rule was fixed after the merge's reserve scores had been read, so only its TEST part (+0.2 points, 95% CI −6.5 to +7.5) is strictly prospective, and all our registrations are timestamped entries in our own repository, not a public registry. The semantic audits and "independent" reviews are LLM agents, not validated against human raters, and some protocol decisions were taken by an LLM coordinator. The mechanistic analyses are exploratory, select and estimate on the same few contexts, and use artificial teacher-forced inputs. All results concern one model, one benchmark and one harness, whose regex misses 5 correct infeasibility answers on TEST. The Claude judge is the unpinned CLI alias `sonnet`; Holo4-27B is `Hcompany/Holo4-27B` at revision `4de03cfe`.

Broader impact. False success claims mislead users and downstream automation; the verifier we evaluate sends desktop screenshots to a third-party model, a privacy cost; our results argue for external verification and for caution with interpretability-based repairs. We release no new model.

References

[1] 1azybug. RealtimeGUI: a decoupled computer-use agent framework with a holo-3.1 OSWorld reproduction. GitHub repository, commit a763361, 2026. URL https://github.com/1azybug/RealtimeGUI.

[2] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction, 2024.

[3] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances, 2021.

[4] Yuqi Chen, Vincent Siu, Yang Liu, Dawn Song, and Chenguang Wang. Tunable tool-call rates in LLM agents via representation steering, 2026.

[5] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, et al. The BrowserGym ecosystem for web agent research, 2024.

[6] DaveTseng2019. Agent keeps acting and reports success after screen capture fails (UAC secure desktop). GitHub issue #15, hcompai/holo-desktop-cli, opened August 3, 2026. URL https://github.com/hcompai/holo-desktop-cli/issues/15.

[7] Chandram Dutta. The termination circuit: how reasoning models stop thinking. LessWrong, July 11, 2026. URL https://www.lesswrong.com/posts/ajhzc6ktEKyFeJFBS/the-termination-circuit-how-reasoning-models-stop-thinking.

[8] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979. doi: 10.1214/aos/1176344552.

[9] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Jie Shi, Xiang Wang, Xiangnan He, and Tat-Seng Chua. AlphaEdit: Null-space constrained knowledge editing for language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[10] H Company. Holo3.1: Fast & local computer-use agents. Blog post, June 1, 2026. URL https://hcompany.ai/holo3.1.

[11] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching, 2024.

[12] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks, 2019.

[13] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.

[14] Zhaoyuan Huang, Tianjie Ju, Pengzhou Cheng, Zheng Wu, Yansi Li, Chuanbiao Song, Jun Lan, Huijia Zhu, Weiqiang Wang, and Zhuosheng Zhang. Do GUI agents know when not to act? Enabling conflict-aware termination for multimodal GUI agents, 2026.

[15] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

[16] Nick Jiang, Isaac Kauvar, and Jack Lindsey. The value axis: Language models encode whether they're on the right track, 2026.

[17] Anton Korznikov, Andrey Galichin, Alexey Dontsov, Oleg Y. Rogov, Ivan Oseledets, and Elena Tutubalina. The rogue scalpel: Activation steering compromises LLM safety, 2025.

[18] Dharshan Kumaran. Reported confidence in LLMs tracks commitment more than correctness, 2026.

[19] Dharshan Kumaran, Viorica Patraucean, Simon Osindero, Petar Veličković, and Nathaniel Daw. How LLMs detect and correct their own errors: The role of internal confidence signals, 2026.

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[21] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model, 2023.

[22] Yanhang Li, Zhichao Fan, and Zexin Zhuang. When AUC 0.998 is not enough: A candidate evaluation protocol for hidden-state probes of indirect prompt injection in multimodal computer-use agents, 2026.

[23] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration, 2018. ICLR 2018.

[24] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, 2023.

[25] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT, 2022.

[26] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.

[27] Aayush Mishra, Daniel Khashabi, and Anqi Liu. Steered LLM activations are non-surjective, 2026.

[28] OnAnOrange. CUA-Lite-Qwen3.5-4B-ScaleCUA-gpt5_5-epoch3-public. Hugging Face model card, 2026. URL https://huggingface.co/OnAnOrange/CUA-Lite-Qwen3.5-4B-ScaleCUA-gpt5_5-epoch3-public.

[29] Jiayi Pan, Yichi Zhang, Nicholas Tomlin, Yifei Zhou, Sergey Levine, and Alane Suhr. Autonomous evaluation and refinement of digital agents. In *Conference on Language Modeling (COLM)*, 2024.

[30] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition, 2023.

[31] Thilo Reintjes, Sivajeet Chand, Derui Zhu, Sushant Kumar Pandey, and Alexander Pretschner. LegacyWorld: Atomicity-aware evaluation of GUI agents for legacy workflows, 2026.

[32] Subramanyam Sahoo, Vinija Jain, Aman Chadha, and Divya Chaudhary. Linear probes detect task format, not reasoning mode in language model hidden states, 2026.

[33] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of Bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 3135–3144, 2017.

[34] Patrick Queiroz Da Silva, Hari Sethuraman, Dheeraj Rajagopal, Hannaneh Hajishirzi, and Sachin Kumar. Steering off course: Reliability challenges in steering language models, 2025.

[35] Yuan Sui, Yulin Chen, Yibo Li, Xue Jiang, Yufei He, Yihong Dong, Xiaoxin He, Tianyu Gao, and Bryan Hooi. TACT: Mitigating overthinking and overacting in coding agents via activation steering, 2026.

[36] Chung-En Sun, Linbo Liu, Ge Yan, Zimo Wang, and Tsui-Wei Weng. LLM agents already know when to call tools – even without reasoning, 2026.

[37] Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Blog post, MIT CSAIL, November 2, 2026. URL https://rikiyat.github.io/blog/weight-space-programming/.

[38] Daniel Tan, David Chanin, Aengus Lynch, Dimitrios Kanoulas, Brooks Paige, Adrià Garriga-Alonso, and Robert Kirk. Analyzing the generalization and reliability of steering vectors, 2024.

[39] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering, 2023.

[40] Constantin Venhoff, Iván Arcuschin, Philip Torr, Arthur Conmy, and Neel Nanda. Understanding reasoning in thinking language models via steering vectors, 2025.

[41] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small, 2022.

[42] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[43] Zekun Wu, Ze Wang, Seonglae Cho, Yufei Yang, Adriano Koshiyama, Sahan Bulathwela, and Maria Perez-Ortiz. Tool calling is linearly readable and steerable in language models, 2026.

[44] Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. AxBench: Steering LLMs? Even simple baselines outperform sparse autoencoders, 2025.

[45] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.

[46] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.

[47] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

[48] Mohammad Zbeeb, Hasan Abed Al Kader Hammoud, and Bernard Ghanem. Reasoning vectors: Transferring chain-of-thought capabilities via task arithmetic, 2025.

[49] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods, 2023.

[50] Chengfeng Zhao, Yuqiao Tan, Shizhu He, Yequan Wang, Jun Zhao, and Kang Liu. Neural procedural memory: Empowering LLM agents with implicit activation steering, 2026.

[51] Tianyi Zhao, Yinhan He, Wendy Zheng, Yujie Zhang, and Chen Chen. Wired for overconfidence: A mechanistic perspective on inflated verbalized confidence in LLMs, 2026.

[52] Chujie Zheng, Ziqi Wang, Heng Ji, Minlie Huang, and Nanyun Peng. Model extrapolation expedites alignment, 2024.

[53] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, et al. Representation engineering: A top-down approach to AI transparency, 2023.

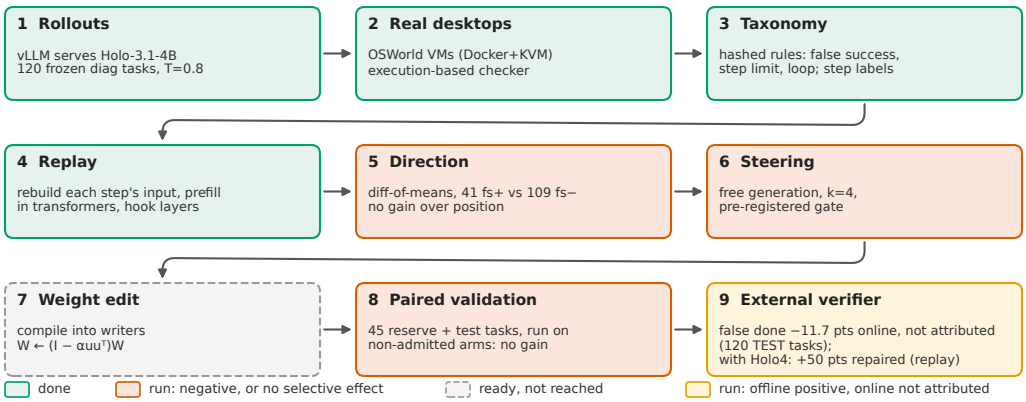


Figure 5: The pipeline and where it stands. Stages 1–4 are complete and verified. The official direction analysis (5), on 41 admissible FS+ steps, finds no signal beyond position. Steering (6) has been run, and no candidate passes the gate. The compiled weight edit (7) is built and tested on null patches, but not reached because no candidate has been admitted. The paired online validation (8) was run anyway, on arms that had not been admitted (§F.6), and shows no gain beyond the run-to-run noise of the original. An external verifier (9, §G) separates false from true successes offline; online, on the 120 TEST tasks, the guarded run had fewer false "done" (−11.7 points) with no measurable score change, but most of that drop came from episodes in which the guard never ran, so it is not attributed to the guard; handing its refusals to Holo4-27B adds +50 points of repairs (95% CI +22 to +78, 9 episodes), measured in replay only.

A About the appendices

The main text condenses the analyses below. Appendices B–L give each of them in full, with every number, the remaining figures and tables, the compilation of interventions into weights (Proposition 1) and the 0.8B study; figures and tables already shown in the main text are referenced rather than repeated. Figure 5 summarises the pipeline and where it stands.

B Setting

Models. Holo-3.1 models are fine-tunes of the Qwen family [10]; their released configuration is a Qwen3.5 vision-language model. Their decoder is hybrid: three linear-attention layers (Gated DeltaNet [46]) for every full-attention layer, and tied input and output embeddings. Holo-3.1-4B has 32 layers of width 2560 (MLP width 9216), 24 of them Gated DeltaNet and 8 full attention; Holo-3.1-0.8B has 24 layers of width 1024. Layers are indexed from zero. At each step the agent writes one JSON object whose fields come in a fixed order, a `note`, a `thought`, then the `tool_call`, after its reasoning.

OSWorld harness (4B). We reuse the Holo agent loop as packaged by RealtimeGUI at commit `a763361` [1], a third-party reproduction rather than an H Company repository, changing only the model provider. Every run uses its default system prompt (variant `v2`, recorded in each episode's metadata), which is RealtimeGUI's, not H Company's, and already asks the agent to check each action on the new screenshot, to "never assume success" and to call `answer` only once it has "verified the result on screen". False "done" is therefore measured under a prompt that already asks for verification. Its rule for recognising a declared infeasibility (`task.{0,4}infeasible|infeasible|cannot be (completed|done)|not possible`) is broader than that of the official OSWorld adapter for Holo, which accepts only `task.{0,4}infeasible`; our absolute scores are therefore not strictly comparable with H Company's, while paired comparisons within our harness are. vLLM 0.23 serves the 4B with thinking enabled, $T = 0.8$, at most 50 steps and the three latest screenshots in context. OSWorld [45] tasks run in Ubuntu VMs under Docker+KVM, at most four in parallel, and are scored by the benchmark's execution-based checkers. We fixed three infrastructure issues in the VM provider (a memory cap, binding VM ports to localhost, an nftables network script) and modified no task or checker ourselves. The OSWorld copy inside RealtimeGUI, which every local and GitHub run used,

is not byte-identical to upstream: its Chrome evaluator getters wait for page load with a fallback (`networkidle`, then `load`, then `domcontentloaded`) instead of `networkidle` alone, a change its authors made for a slow proxy. Checkers of Chrome tasks that fetch a web page may therefore score differently from upstream OSWorld.

Frozen split. From the 361 tasks of RealtimeGUI's `test_nogdrive.json`, we sort ids by SHA-1 within each domain and deal them alternately into a DIAG and a TEST pool, keeping 120 tasks each with per-domain quotas. The split files have SHA-1 prefixes `51e670b3` (diag) and `957d68e7` (test). No construction or screening read the test split. Six valid online runs then used it: five model runs (§F.6), the original, served once by our H200 server and once by Modal, the ExPO edit, which went to it without having passed the gate, a partial corrective-prompt run, and the CUA-Lite merge under a decision rule fixed after its reserve scores had been seen; and the guarded agent (§G). A seventh run, of an output-head scaling, was invalidated (§F.6).

MiniWoB++ harness (0.8B). Our first attempt used Holo-3.1-0.8B on MiniWoB++ [23, 33] through BrowserGym [5], with the task area cropped and upscaled $5\times$; it is described in §H and Appendix R.

Hardware and licences. Rollouts, replays and screens ran on one NVIDIA H200 shared between the vLLM server and research jobs; edited checkpoints were also served on rented H100 GPUs, and the OSWorld VMs ran on a separate CPU host. As order-of-magnitude estimates, the project used about 14 hours of one H200 (rented, shared by every research job and the rollout server, so per-experiment GPU-hours are not separable), about 8 H100 GPU-hours (USD 31.9 billed over two Modal accounts by 05:11 UTC) and about 60 GitHub Actions CPU jobs for the OSWorld VMs. These figures stop at 05:11 UTC: the later Modal deployments (the Holo4-27B judge and agent, the Holo-3.1 model behind the guarded agent) and the Claude calls of the verifier are not included. Holo-3.1-0.8B and -4B, Qwen3.5-4B, the CUA-Lite donor, OSWorld and BrowserGym are released under Apache-2.0 and MiniWoB++ under MIT; the RealtimeGUI repository has no licence file at the cited commit, and we use it unmodified except for the model provider.

C Most failures are a false "done"

Taxonomy. Each episode gets one primary category from fixed rules over its actions, screenshots and final score. *False success*: the agent calls `answer` on a feasible task scored 0, or claims completion on an infeasible task. *Loop*: at least three consecutive identical actions while less than 1% of pixels change. *Step limit*: the 50-step budget is exhausted without a loop. At step level, FS+ marks the final `answer` of a false success and FS− that of a true one. A correctly worded refusal of an infeasible task is excluded from both. The rule file was hashed before the official direction analysis (current hash `1e5079afb500`); later changes are logged with their hash. One of them, made before any OSWorld AUROC, altered a label: an infeasible task answered with a plainly worded refusal (`gimp/38f48d40`) moved from FS+ to excluded. Before the official direction analysis (exploratory directions had been fitted on smaller label sets) we also require that the target category cover at least 25% of failures and provide at least 40 positive steps (the GO rule).

Result. On the diagnostic split (120 tasks, 119 scored), Holo-3.1-4B scores 56.7% (the official OSWorld score, partial credit included, the convention used for every score in this paper) and fully solves 66 tasks: 55.5%, 95% Wilson CI 46.5–64.1. An independent reproduction with the same public harness reports about 45.5% at 100 steps on a different task set [1]; we do not claim the two numbers are comparable. Of the 51 failures (Figure 6), 29 (57%) are false successes, 16 (31%) have the step limit as primary category (21, or 41%, exhaust the 50 steps at all, loops included), 5 (10%) loop, and 1 is unclassified; 1 further task is unscored (the "Other + unsc." column of Table 9 counts both). 9 false successes are claims of completion on infeasible tasks, and 20 are feasible tasks done wrongly or partially. On an infeasible task the correct action is `answer` with a refusal, so $P(\texttt{answer})$ is the wrong metric there; one such context, Chrome `480bcfea` at step 4, enters several screens below, and Appendix M gives every count with and without it. These are mostly not misworded refusals: among the 75 tasks open to construction, 6 are infeasible (11 episodes), and of the 10 final answers 8 claim success, while only 1 task states its inability, in words the harness rule misses. The category is wider than explicit claims of completion. Read strictly, three of the 9

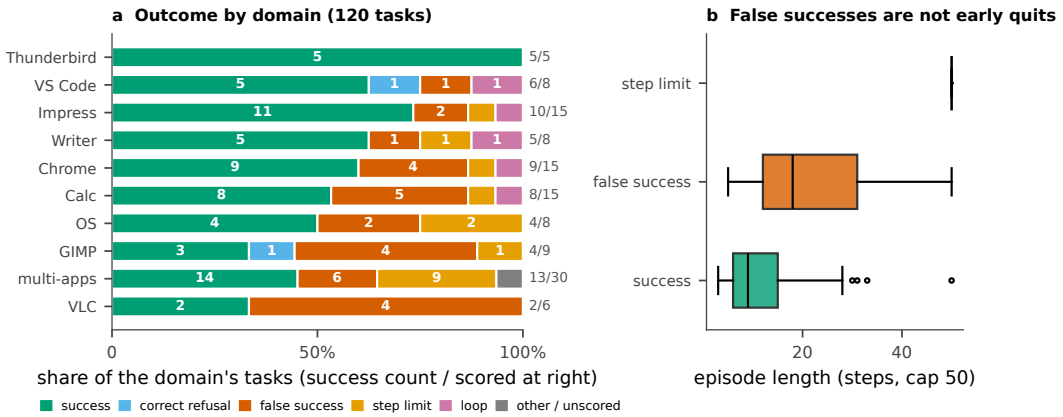


Figure 6: False success is spread across domains and is not an early quit. Holo-3.1-4B, OSWorld diagnostic split, 50 steps, $T = 0.8$. (a) Primary outcome per domain, sorted by success rate; numbers in bars are task counts, and the right column gives fully solved tasks over scored tasks (the *success* category also counts the 2 partially scored tasks). (b) Episode length by outcome. The full table is Appendix Table 9.

infeasible false successes of the diagnostic split do not claim completion: two decline the task in words the harness rule misses (`gimp/38f48d40`, `vs_code/7c4cc09e`) and one states that it cannot finish and asks for the user's credentials (`writer/bb8ccc78`). Of the 20 feasible ones, 19 final answers claim completion and one (`multi_apps/b337d106`) presents a tutorial instead of applying the setting it describes, on our reading, done by an LLM agent and not validated by humans. The strict count of explicit false claims of completion is therefore 25 of 51 failures (49%); of the other four, two are correct refusals the rule misses and two are answers given instead of finishing, like the episode of Fig. 1a. They are not early quits: the median false success ends after 18 steps. Every domain has false successes except Thunderbird, and Chrome, Calc, GIMP and VLC have four or more each (Fig. 6a). False success passes the share criterion of the GO rule, but its 27 FS+ steps alone do not pass the count criterion. With a second pass over the same tasks, stopped at 80 scored tasks when the GPU returned to production, 41 FS+ and 109 FS− steps are admissible, above the required 40 (as of 2026-11-07 21:56:45 UTC).

C.1 The model says "done" on most false completions too

We re-sampled the original model on six screening contexts from five tasks, two per stratum ($k = 4$, production sampling, thinking enabled, invalid outputs counted; Fig. 1c and Table 2, first row). It calls `answer` on 8/8 false-completion samples and 8/8 true ones, and never on mid-episode contexts (0/8). It reproduces the terminal versus non-terminal distinction exactly, and on these contexts shows no false versus true distinction; on the larger, strictly feasible cohort of §F.5, it answers on 72% of false and 94% of true completions. Any repair must therefore change a decision the model takes in most false and nearly all true completions, and only in the false ones.

The reasoning has the same shape in both cases. On 16 re-sampled reasoning traces at completion steps of 4 construction tasks, 8 on false and 8 on true completions, every trace opens with a success claim ("The task has been completed successfully", "The document has been saved successfully") at generated-token index 4–5, counted from zero, with no verification word before it (0/16), then lists the steps taken (median 116 tokens). Because true completions show it too, this is a property of Holo's output format at completion steps, not a marker of false success. It locates the written conclusion, not the moment the decision forms inside the model. A direction fitted on these tokens separates the two pairs perfectly, but so do context length and a single lexical cue, so we give it no weight; steering along it is negative (Table 2). What the format skips is the check: in the Spotify episode, the reasoning states that the installation cannot finish, and the same step calls `answer`, reporting the installation as "initiated successfully" and handing the password step to the user, which ends the episode unfinished (Fig. 1a).

D From a served agent to activations

Rollouts are served by vLLM [20], which does not expose activations. We therefore never record activations during rollouts. For each recorded step we *rebuild* the exact chat the agent sent: system prompt, task, `<observation>` blocks, at most three screenshots with older ones replaced by `[screenshot evicted]`, parsed tool calls and tool outputs. We render it with the model's chat template and run one forward pass in `transformers`, hooking the residual stream and MLP output of every layer at two positions: the last prefill token, and the token before the action JSON. Coverage is partial on OSWorld. To keep up with the runs, 52 of the 119 pass-1 episodes and 61 of the 80 pass-2 episodes were replayed on their last three steps only, and 26 episodes were skipped because a context exceeded the replay memory cap, which removes the longest contexts; in all, 6 of the 47 FS+ steps have no replayed activations.

Fidelity is tested, not assumed. (a) *Messages.* Running the real agent loop with a fake client and environment, the rebuilt messages are identical to those sent on 7/7 steps, also when three consecutive parse failures are injected before steps 1, 4 or 5, the harness's image-eviction edge case. (b) *Tokens.* The prefill length in `transformers` equals vLLM's `prompt_tokens` on 12/12 steps of a real trajectory (4,459 to 10,432 tokens), and the replay loop repeats this check on the first and last step of every task. This test tokenises our own rebuilt messages on both sides, so it validates the image processor and the tokenisation, not the rebuild itself, which test (a) covers. (c) *Likelihood.* The replayed model gives its own recorded actions a mean log-probability of -0.16 per token, against -1.10 for synthetic actions. One divergence is invisible in the files: an exception in `env.step` leaves an unrecorded assistant message in the live context. The diagnostic pass logged 0 such events and 6 triple parse failures, which test (a) shows to be harmless.

E Three traps that would each have produced a positive result

Trap 1: the direction measures position. False completions come later in an episode than true ones, so anything that grows with the episode separates them. A first difference-of-means direction between FS+ and FS− steps (22 and 55 steps), cross-validated by task, peaked at AUROC 0.836 against 0.870 for the step index and 0.880 for context length. Reported without baselines, 0.836 reads as a solid detector; in fact it was worse than counting steps. The official analysis, run once FS+ steps passed the pre-registered 40 (41 FS+, 109 FS−, 95 tasks, both passes; exploratory because the second pass stopped early), finds no more than position (Table 1, Fig. 2b). The best direction, at layer 14 on the last prefill token, reaches 0.76 (logistic probe 0.71) against 0.724 and 0.735 for the two position baselines, within noise of them (standard error 0.05), and the token before the action does worse (0.70). Controlling for position directly, a logistic model on step and log context length scores 0.722; adding the best direction raises it by at most 0.04, at a layer chosen after the fact, within noise. We find no false-success signal in the residual stream beyond the length of the episode, which is consistent with the failure of every direction-based intervention below. A value-axis direction in the style of Jiang et al. [16] shows the same confound in a separate construction (appendix Q). Our protocol therefore matches positives and negatives on step and context length before fitting, and reports both baselines next to every AUROC.

Trap 2: regression to the mean. Agent outcomes are stochastic at $T = 0.8$. Selecting tasks because they failed in one run selects partly for bad luck. Re-running the unmodified agent on the same tasks, only 9/18 (50%, 95% Wilson CI [42] 29–71) pass-1 false successes are false successes again (an earlier snapshot of the second pass, on fewer tasks, gave 42%), and 6 of them now succeed (Fig. 2c). A patch evaluated as "false-success tasks before versus after" would therefore be credited with about 50 points of reduction while doing nothing. Our validation compares two fresh runs, original and patched, on a frozen reserve of 45 tasks that no intervention was built on (§E.1).

Trap 3: detecting is not causing. The 0.8B loop direction is a textbook detection result (Fig. 2a): task-grouped AUROC 0.893 at layer 10, against 0.70 for the step index and 0.72 for context length, replicated at 0.904 on an independent refit. It fails every causal test we ran: it reduces loops only at a large cost to healthy actions (§H). Projecting it out of every writer erases the marker from the residual stream (the loop/non-loop projection gap at layer 10 falls from 0.1653 to -0.00008) and

leaves loop actions as likely as before relative to healthy ones. The marker can be deleted without touching the behaviour it marks.

E.1 The protocol these traps impose

1. **Free generation is the only efficacy measure.** Teacher-forced log-probabilities of a recorded action mix format, arguments and end-of-sequence, and on the 4B the decision is taken during reasoning, after which $P(\texttt{answer})$ is close to 1 for true and false completions alike. Each candidate regenerates the full reasoning ($k = 4$ per context, fixed seeds shared across arms, production sampling with `top_k=20`), and invalid or truncated outputs stay in every denominator.
2. **A gate fixed before any data.** On FS+ contexts of tasks not used to build the intervention, $P(\texttt{answer})$ must fall by at least 25 points and 50% relative. The FS− answer rate and the rate of valid non-answer actions on mid-episode contexts may each fall by at most 5 points, and invalid outputs may rise by at most 5 points. Later families (the mechanism-level edits, the conditional circuits and the narrow MLP edit of Appendix N) froze a looser bound of 10 points on true completions and healthy actions. The FS− criterion is what separates "less false success" from "less success".
3. **Controls.** A random direction with the same norm and scope, an identity hook that measures the noise floor, and a fixed budget of generation batches per hypothesis, counted before execution, failures included. Two consecutive settings without progress stop a hypothesis.
4. **Disjoint confirmation.** The official direction analysis is gated at 40 admissible FS+ steps. The 45-task confirmation reserve is excluded from the construction and screening of every intervention, with one declared exception (a stuck-state direction allowed to use half of it, which produced no candidate), and was to be read only to confirm a committed candidate, with both arms re-run fresh. It is not excluded from the descriptive direction analysis: 20 of its tasks are among the 95 of Table 1.

F No selective effect found

F.1 Where the tool choice is carried

Before steering, we mapped where the choice of `answer` is made. On two FS+ and two FS− completion steps, we patched the residual stream from the same task's previous step (whose recorded action is a click) at each of 32 layers and four positions, 512 patches in all, with the original reasoning held fixed. The quantity is the logit margin between the first token of `answer` and that of the best other tool. Identity patches give exactly zero.

Because the reasoning is forced to its recorded text, a patch before or inside it cannot change what the model writes, so the decision is invisible at the end of the prefill and barely visible mid-reasoning by construction: the largest mean margin change is -0.19 and -2.28 logits there, for FS+. It appears only at the tool-name position, from about layer 16 upward, where patching moves the margin by up to -34.8 logits on FS+ and -37.5 on FS− (Fig. 3). Component scans single out three attention heads, L19/H15, L23/H1 and L31/H14, and ten late MLP neurons. Their attention falls mostly on the system-prompt definition of `answer` (L19/H15, L23/H1) and on the JSON being written (L31/H14). Patching all of them together from the donor step lowers the margin by 11.2–12.2 logits, and flips 0 of the 4 decisions (smallest remaining margin 2.9). Across all 992 component patches and 28 joint patches, 0 and 0 decisions flip: the components move the margin, not the action. The only flips come from residual patches (21 on FS+ and 23 on FS− of 512), at the tool name in the last layers, where replacing the residual stream transplants the donor's choice into the output head. They show where the choice is carried, not where a false judgement is formed. The effect is the same on true and false completions (Fig. 3c). On these four contexts, one of them infeasible, and with component scans limited to layers 19, 23, 30 and 31, these components behave like a tool-name retrieval circuit, not a false-success circuit.

A ceiling test. The patches above replace one layer at a time; a ceiling test asks how much must be replaced at once. With the recorded input forced and only logits read, the outputs of whole blocks of writers are swapped in at the decision position from a donor step where the model clicked: the same task's previous step for two FS+ contexts, another task's step for one true completion (Fig. 4). On the first FS+ context (margin +16.9), all 32 MLPs together bring the margin to +0.9 without flipping

it, while the MLPs and token mixers (attention and DeltaNet) of layers 12–23 flip it (-3.5), as do those of layers 24–31 (-10.0), and those of layers 0–11 do not (+16.1). The second FS+ context behaves the same (MLPs +2.1, block 12–23 -3.0), and so does the true completion (block 12–23: from +16.4 to -2.2). Swapping with itself and swapping the final residual reproduce the baseline and the donor exactly. Three caveats bound this result. The first FS+ context is an infeasible task, on which `answer` (with a refusal) is the correct tool (Appendix M). The donor differs in screen, reasoning, thought and position, not only in decision. And since a true completion flips too, the test locates where the identity of the next tool token is assembled, not a mechanism of false success. No particular block is needed: the MLPs and mixers of layers 12–23 or of layers 24–31 each flip the first context, and removing a single direction of the swap difference from the writers of layers 12–23 brings the margins of the two FS+ contexts to +1.4 and +1.6, but also those of two true completions to +4.8 and +2.8. It is consistent with the failure of every edit confined to one or two layers or to the MLPs, and it says nothing about which edit would separate false from true completions.

F.2 The tool call leans on the text already written

The tool call leans on the text the model has already written, for true completions as for false ones. On the 19 false and 20 true completions of the strictly feasible cohort (§F.5), teacher-forced with the screen, history, reasoning and token count fixed, replacing the note and thought by a neutral text of the same token length ("Observing the current screen.") removes 14.0 logits of `answer` margin on false completions (95% CI 12.9–15.1; original 17.3) and 14.5 on true ones (12.9–16.0; original 18.0), 82% and 80% of the margin per task. The two losses differ by -0.46 logits (-2.36 to +1.50; label permutation $p = 0.65$). A counterfactual text of the same length ("The task is not complete yet; I still need to inspect the screen.") leaves 0.10 and 0.61 logits, nearly cancelling the margin on average (`answer` still leads in 10/19 and 13/20 contexts), while neutralising the recorded reasoning instead removes only 4.1 and 4.8. One true completion has an empty note; without it, the true-completion loss is 15.0. The written note and thought thus carry 80–82% of the `answer` margin in both strata, yet without them `answer` remains the preferred tool in 37 of 39 contexts (18/19 false, 19/20 true): the conclusion is reinforced by the note, not created by it, although the success claim in the retained reasoning may carry the rest. This is an attenuation of the margin, not a change of decision. Nor does a probe find a robust discriminator in the residual stream of these contexts: of 72 out-of-fold probes (6 layers, three positions, four models), the best, chosen after the fact, reaches AUROC 0.64 (0.47–0.80) against 0.70 for the raw step index, and its permutation p of 0.04 becomes 1.0 after Bonferroni correction for the 72 probes. With 1,000 permutations the corrected p cannot fall below 0.072, so this corrected test had no power at all. With these sample sizes a pre-specified score would need an AUROC of at least 0.725 (one-sided) to 0.755 (two-sided) to be detected with 80% power, and about 0.85 after a correction for 72 probes, so the absence of the information is not proven. These are artificial inputs, and the joint replacement does not separate the wording of the conclusion from the details the note describes.

A first version of the test, on two FS+ contexts, illustrates it with a donor's text rather than a neutral one: it keeps the screen, the history and the weights, and replaces only the text the model has generated at the decision step with that of the same task's previous step, where the model clicked. On the first FS+ context (margin +16.9), the donor's reasoning alone lowers the margin to +7.6, the donor's JSON prefix alone (note and thought) to +1.1, and both together flip it (-7.4). The second context behaves the same (+18.4; reasoning +11.1, JSON prefix -3.1, both -7.6). Three limits weighed on that version, the first of which the cohort test above removes: it had no control on true completions or with a neutral text. Only its second context is feasible, and the donor is the previous step, whose text announced a click, so that copying its text largely copies its decision. And the text does not account for everything: with all of it swapped, the margin stays 9.8 and 14.5 logits above the donor's, a share that comes from the screen and the history. Within these limits, the tool call depends causally on the text already written, the note and thought weighing most, and the 13.9–19.1-logit margin of §F.3 is largely conditioned on that text rather than an intrinsic robustness of the decision. An attribution on the same cohort locates the channel. Replacing the inner tokens of the note and thought with spaces, at the same length and positions and with the screen, history and reasoning kept, lowers the teacher-forced margin by 7.55 logits on false completions (mean 17.3 before) and by 8.18 on true ones (mean 18.1 before, measured on its own cached replay path), less than the neutral text above, and lowers it on every context of both (19/19 and 20/20); the difference between the two losses is -0.64 logits (95% CI -2.13 to +0.80). The note and thought carry the

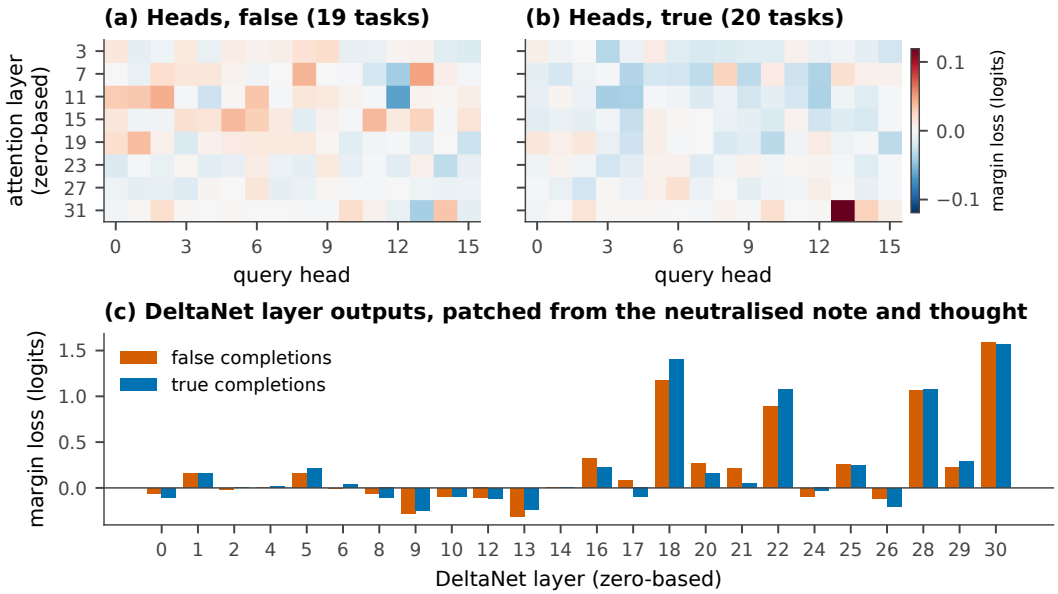


Figure 7: Single DeltaNet layer outputs carry the written conclusion to the tool choice more than single attention heads do, for true completions as for false ones. Teacher-forced `answer` margin lost when the note and thought are neutralised, on the strictly feasible cohort. (a, b) Blocking the final query's attention to the note and thought, one head at a time: at most 0.12 logits (colour scale). (c) Replacing one DeltaNet layer's output at the decision position by its value under the neutralised note and thought: 0.9–1.6 logits at layers 18, 22, 28 and 30, alike for both strata (note the ten times larger scale). Single-layer patches do not add up to the whole effect (7.55 and 8.18 logits). A whole-layer output patch includes upstream effects that a single-head block does not, and no full-attention layer output was patched as a control. Heads and layers are screened on the same cohort: exploratory.

decision as much for true as for false completions: the channel is real, but not selective. Two kinds of patch probe the route (Fig. 7). Replacing the output of a single DeltaNet layer at the decision position by its value under the neutralised text costs 0.9–1.6 logits at layers 18, 22, 28 and 30 (zero-based) and at most 0.33 at the other layers. Blocking the final query's attention to these tokens, one head at a time over the 128 heads of the eight full-attention layers, costs at most 0.048 logits on false completions and 0.12 on true ones, and at best 0.05 logits more on false than on true. The two tests are not symmetric: a whole-layer output patch includes every upstream effect on that layer, while a head block cuts one attention link of the final query, and no whole full-attention layer was patched as a control. So single DeltaNet layer outputs carry 0.9–1.6 logits each and blocking the final query's attention one head at a time removes at most 0.12; this does not show that the recurrent memory rather than attention carries the conclusion. The same contexts define the comparison and the cohort, so this screen is exploratory, and layer-output patches locate carriers of the change, not its mechanism.

F.3 Every steering arm

Table 2 lists every intervention run on the 4B false-success target, each under the gate of §E.1 and on six screening contexts, the same six for every arm except the neuron and `perceive` arms, which use their own. Each line tests a distinct hypothesis; four come from the literature, the others from our own analyses.

- *Tool choice is linearly steerable* [43], tested with the position-matched FS+−FS− direction rather than a tool-choice direction: add it at the last prefill token with coefficient $\alpha = -1.20$ at layer 16 and $\alpha = -5.54$ at layer 24 (absolute magnitudes; a negative α subtracts the direction, and Table 2 gives the dose relative to the residual norm).
- *The decision is taken during reasoning* (§C.1): subtract a "conclusion" direction fitted on the reasoning tokens ($\alpha = -0.5$) at layer 31, at every generated token, including the JSON after `</think>`, so the arm can also act on the tool choice directly.

- *False success is excess expected value*, a hypothesis derived from Jiang et al. [16] and Kumaran [18]: push toward low value or low commitment at layer 24 or 18.
- *Late MLP components write the decision* [7], tested here at the neuron level: zero the top-16 or top-64 neurons by direct attribution to the `answer` logit.
- *The model fails to check*: add a "verify" versus "conclude" direction (contrastive activation addition, 30) at layer 18 at 1 and 2 times the residual norm, over the prefill and the first 20 generated tokens.
- *False success is a perception of completion*: subtract the imminent-completion direction of §F.4 at layer 15.
- *The model opens with its conclusion* (§C.1): raise the output row of the token "Wait" for the states that open a completion, a 2560-weight edit of the output head.
- *Memory of the early episode fades*: lengthen or shorten the retention of the nine late linear-attention layers by a factor 1.25.
- *The circuit of §F.1 carries it*: scale its three heads and ten neurons (C1) by $1 - \alpha$ with $\alpha = 0.5$ and 1, against a random direction of the same contribution norm.

No arm passes the gate, and none shows a large effect. Most leave the false-"done" rate at its original value, including removing the whole C1 circuit ($\alpha = 1$), which answers on every false completion, like the random control. The neuron arms, screened on six other disjoint contexts whose own baseline is 6/8 FS+ answers, were stopped by a mathematical bound: with 4/4 answers already observed on the first unseen FS+ task, at most 3 answers out of 8 would have been admissible. The value-axis arm did change the text. An LLM-agent reader found an explicit intention to re-evaluate in 3/8 FS+ reasoning traces against 1/8 at baseline, but 0 verification actions and the same answer rate. The model wrote that it should check, then answered anyway.

The "Wait" row is the one edit that lowers false answers without touching anything else: 6/8 on false completions, with 8/8 true completions kept, 8/8 healthy actions valid and 3/3 general-text controls token-identical. It still fails the 50% relative criterion, and an LLM-agent review rates none of the two replacement actions useful: one reloads a settings page for an invented reason, the other scrolls without fixing the dates. Making the model pause is not enough to make it repair. A conditional variant adds an early end-of-reasoning row to the same edit (5101 weights). Inside its construction set, 5/8 false completions still answer. On one Budget sample the model reasons correctly that next month is December, says the dates must be changed, and generates a click on the date summary of the results page; it was not executed. These are intentions to correct, not corrections: an LLM-agent review of the screenshot finds that the dates sit in plain summary text, next to a separate "Modify Rental Details" control, and that no step proves the summary opens an editor. All 7 such generated clicks, over 4 lots of this edit and its diagnostics, are reclassified as unverified, 0 confirmed useful; no effect on the interface or on the OSWorld score is shown. Nor does the edit transfer. On nine training tasks left out of its construction (8 of the 12 FS+ outputs come from two infeasible tasks), the final audit counts 0/12 useful actions on false completions (original 0/12), 3/12 outputs that name the real error (original 1/12), and 7/12 false finishes (original 8/12), while true completions go from 10/12 to 9/12. The edit leaves a mechanistic signal, the model recognising the error then rationalising it away, and no demonstrated useful effect.

These screens are pilots. Each efficacy cell holds 8 generations from two tasks. 8/8 has a Wilson 95% interval [42] of [68, 100]%, and samples clustered in two tasks make the real uncertainty larger, so a rejection means only that no large effect appears on these contexts, not that none exists. By design the gate flags a reduction to at most 4/8 answers; smaller effects pass unnoticed. Table 2 gives the interval of every efficacy cell.

A ceiling effect is likely. At the tool-name position the margin of `answer` over the best other tool is 13.9–19.1 logits on false completions and 10.9–17.8 on true ones. This position comes after the JSON `thought` field, which the output schema writes before the tool call and which already states that the task is complete, so the margin measures how confidently the model carries forward a decision it has already verbalised, not the margin of the decision itself. Even so, the perturbations tested are small: next to the residual stream, they are 0.13 (layer 16) and 0.23 (layer 24) of the mean residual norm for the matched direction, 0.25 for the reasoning-token direction (difference-of-means norm 24.5 against a residual norm of 50.0 at layer 31), and 0.5 for the value axes, while patching the whole C1 circuit from the donor step lowers the margin by 11.2–12.2 logits without flipping it. Larger doses do not break the ceiling either. The "verify" direction at one and two times the residual

norm leaves FS+ at 8/8; at dose 2 a healthy context loops on "I see" up to the 3,072-token cap, which stopped the arm after 23 of 24 outputs. Dose sweeps at layers 24 and 31 with random controls were not run.

A second target, getting stuck at the step limit, shows the opposite failure of steering. A residual addition at layer 16 or 24 damaged healthy actions before any stuck context was generated; after 5/24 outputs, the best reachable health rate was 91.7%, below the required 95%.

F.4 A detector of imminent completion beats position, not false success

All directions above contrast false and true completions, which differ in position. A separate analysis sidesteps this by learning a direction *inside successful episodes*: the last-prefill residual of the final, correct `answer` step minus that of the step just before it, on 58 pairs from 40 construction tasks (reserve excluded, cross-validation grouped by task across both passes). Because both members of a pair sit next to each other in the same episode, position cannot separate them across tasks. The direction peaks at layer 15 with task-grouped AUROC 0.910, against 0.552 for the step index and 0.555 for context length (Fig. 8a). This is the first signal in this study that position does not explain, in contrast to the false-success direction of Table 1. Within a pair, both baselines are trivially perfect, so the pooled AUROC is the relevant comparison, and the layer was chosen on the same cross-validation.

Read on 14 false/true completion pairs matched on pass, step and context length, with each task's direction fitted without that task (3 of the 14 false completions are infeasible tasks, and one false-completion task appears in two pairs), the direction rises at the "done" step in 14/14 false and 14/14 true completions (mean 1.11 and 1.54 standard deviations; Fig. 8b). At the terminal step, false completions sit -0.14 standard deviations from true ones, above them in only 6/14 pairs. It is therefore a detector of imminent completion, not of false success: it fires before a false "done" as it does before a true one. On these 14 pairs, the projection reads a false completion as complete. This is compatible with a perception error: the screen is taken as showing a finished task. Two same-task image pairs from construction data illustrate it, a spreadsheet cell claimed green whose text has no green pixels, and rental dates claimed for next month while the page shows the current one. It does not separate misperception from commitment to the action. Subtracting it at layer 15 (up to 0.68 of the residual norm, on its own screening contexts whose reference is 6/8 FS+ answers; both FS+ contexts are infeasible tasks, so this screen tests no feasible false success) gives 5/4/5/5 FS+ answers out of 8 for $\alpha = 0.5, 1, 2, 4$, against 7/8 for a random direction of the same norm, and no dose passes the gate (Table 2). The few drops are not repairs: the non-answer actions that replace `answer` are empty or invented text entries, such as a made-up e-mail address. Attention maps on two same-task pairs found no head, out of 80 tested, that reads the evidence region more when the model succeeds (0/80 pass the frozen rule). The perception hypothesis is not refuted by this small screen, but it has no support beyond the projection of Fig. 8.

F.5 Weight edits: levers without established selectivity

Weight edits that need no direction, or that target a mechanism, are detailed in Appendix N; we summarise them here. Extrapolating the fine-tune away from Qwen, $W(\lambda) = W_{\text{Qwen}} + \lambda\,(W_{\text{Holo}} - W_{\text{Qwen}})$, called ExPO [52], leaves false answers at 16/16 in every arm but one (15/16, MLPs only, where true completions drop more); the edit at $\lambda = 1.25$ keeps text and JSON intact and runs online (§F.6). The mechanism-level edits H2–H10 (memory gates, MLPs or all language tensors moved toward Qwen, the end-of-thinking embedding, the attention temperature, the rotary share of the query gain) leave the tool call in place, break generation, or move the call only on the infeasible context, where theme clicks replace an answer that should have explained the impossibility (Table 8); two-neuron circuits, a narrow MLP edit adapted from weight-space programming [37] and a corrective instruction injected into the reasoning do no better. Two edits do move the decision on feasible contexts, and neither shows an established selectivity.

Merging in a donor trained on desktop tasks moves the decision globally, not selectively. Adding a fraction β of the task vector [15] of CUA-Lite, a Qwen3.5-4B fine-tune on ScaleCUA desktop data, to the language layers of Holo ($W_{\text{Holo}} + \beta\,(W_{\text{CUA}} - W_{\text{Qwen}})$; vision tower unchanged; input embeddings merged, the output head being tied to them at load time), and serving it with the same 51 calls as the exact original model, re-served on the same GPU type, image and vLLM version

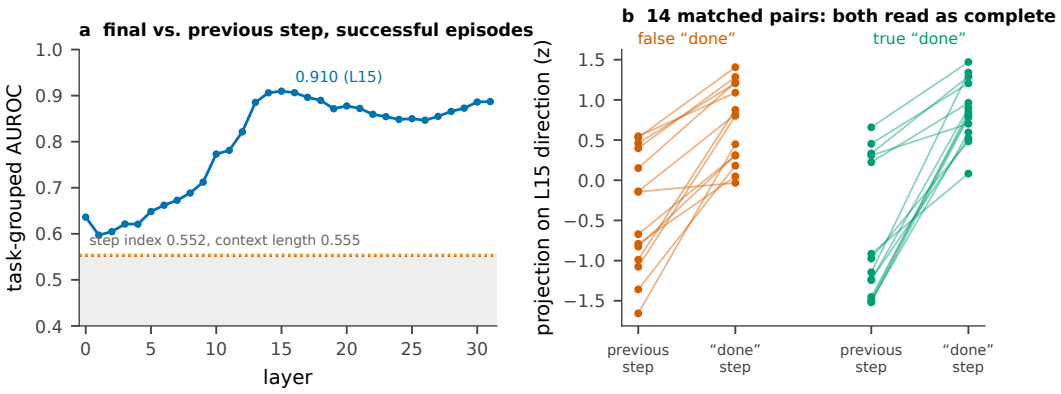


Figure 8: An imminent-completion direction beats position but does not separate false from true completions. Exploratory, no causal test. (a) Task-grouped AUROC of a direction learned inside successful episodes (final `answer` step vs. the step before it, 58 pairs, 40 tasks), by layer; the grey band is at or below the position baselines. (b) Cross-fitted projections at layer 15 on 14 matched pairs: the projection rises at the "done" step for false and true completions alike.

(on a separate Modal account), lowers false-completion answers to 15/16 at $\beta = 0.25$ and 11/16 at $\beta = 0.5$, and true-completion answers further, to 11/16 and 2/16 (original 16/16 and 16/16), with every healthy action valid and no invalid output; on the feasible FS+ context alone, $\beta = 0.5$ answers 3/8. The dose–response is a global drop of `answer`, not a discrimination. An LLM-agent audit finds the replacing actions coherent and unnecessary: scrolls after the model has recognised success, and verification clicks after true completions. The model continues instead of concluding, whatever the truth of the completion. A donor trained on filtered web tasks, Fara1.5-4B, leaves the decision in place at $\beta = 0.5$ (16/16 and 14/16). BF16 rounding blurs small merges: at $\beta = 0.5$, 88 of the 426 language tensors are left unchanged and the update norm is 2.34 against an intended 2.01; at $\beta = 0.25$, 101 tensors are unchanged. CUA-Lite's training data may overlap OSWorld tasks. Online, the merge shows no detectable gain over an original served on the same GPU type, image and vLLM version, from a separate Modal account: +1.9 points over 165 tasks (95% CI −4.3 to +8.0; §F.6).

A closed-form edit of the writers that the ceiling test points to moves the decision as well, unselectively on the screening contexts; on a larger feasible cohort its selectivity is not established either way (below). In the spirit of AlphaEdit [9], the 24 residual writers of layers 12–23 (MLP down-projections and mixer output projections) receive a ridge update that maps FS+ keys (constructed on the ceiling-test contexts, the first of them infeasible) toward the donor displacement while being projected off the span of protected keys (true completions, healthy steps and text prompts). On contexts outside its construction, served next to the original on the same GPU, false-completion answers fall from 15/16 to 8/16 but true-completion answers fall further, from 15/16 to 3/16, with 16/16 healthy actions valid and 0 invalid outputs. The whole FS+ drop comes from one of the two contexts (7/8 to 0/8; the other stays at 8/8). A first visual audit of the replacing actions, which had described the screenshot of the previous step, was retracted; the corrected audit finds that on the e-mail true completion all four recipients were already complete, so the three generated `write` continuations there would potentially damage a correct state (they were not executed). A random update on the same writers, with the same right factors and its per-writer BF16 perturbation norm matched to the learned one within 0.1%, leaves the screening contexts in place (FS+ 16/16, true completions 16/16, healthy 16/16): on these contexts the learned edit moves stopping and this one random draw does not, without selectivity. At half the dose the edit gives 12/16 and 9/16: at both doses, true completions fall on these contexts as fast as or faster than false ones. A 25% dose was not run.

Two distinct methods, a merge and a closed-form edit, thus move the decision. On the screening contexts, true completions fall at least as much as false ones in both, on two true-completion contexts; on the feasible cohort below, this holds for the merge, while the selectivity of the closed-form edit is not established. A simple argument makes a lever without a discriminator unsurprising. An edit applied at or after layer ℓ acts on a function of the residual stream at layer ℓ (all positions, and the recurrent state of the linear-attention layers), so it can treat a false and a true completion differently only to the extent that some function of that stream separates them, and to be useful that function must generalise to tasks it was not built on. Our probes, which read the stream at two positions,

found no such separation beyond episode length (Table 1). A direct test on the writers that both methods edit finds the same. On 19 false and 19 true completions outside the reserve and the evaluation contexts, a ridge probe on the inputs (keys) of the 24 writers of layers 12–23, cross-validated by task, does no better than position. Averaged within test folds and weighted by their 71 positive–negative pairs, its median AUROC is 0.704 (range 0.61–0.90) at the end of the prefill and 0.634 at the end of reasoning, against 0.852 for the same probe on the step index; pooled out of fold, it is 0.596 and 0.496 against 0.644. This pool is weaker than it should be: 5 of its 19 false completions are infeasible tasks, on which `answer` is the correct tool, so the probe mixes in cases where the label marks no wrong decision, which weakens the argument. The strictly feasible cohort below avoids this for behaviour, though not for the probe. The two key subspaces are not identical (median principal angle 67–69°), but 83–86% of the false-completion key energy lies in the true-completion subspace. This is an empirical, non-total overlap together with a strong confound of position and length, not an impossibility: a low AUROC for a linear probe is not an information bound, the probes do not exhaust the stream, and even identical subspaces would not force a selective edit or merge to act globally, which would require proportional covariances.

The agent's own text carries no more. On 66 feasible false and 202 true successes from 90 tasks, the best of 7 detectors reading the final note, thought and reasoning, chosen after the fact, reaches a task-grouped AUROC of 0.66 (95% CI 0.55–0.76) against 0.72 (0.61–0.82) for the final step index; within the 18 tasks that end both ways, every detector lies between 0.46 and 0.54, and a within-task permutation test gives $p = 0.44$. Only the hesitation between samples carries a little: over four samples per context of the feasible cohort below, the original answers at a mean rate of 0.72 on false and 0.94 on true completions, an AUROC of 0.66 (0.53–0.80). Linear probes at two positions found no separation beyond length, and the text detectors none beyond the step index; only the hesitation from one sample to the next carries a little; at these sample sizes this is an absence of evidence, not a proof of absence.

A strictly feasible cohort. To take infeasible contexts out of the evidence, the original, the merge, the closed-form edit and its random control were re-measured on a cohort fixed before any output: 19 feasible false completions, 20 true completions and 16 healthy contexts, all from distinct tasks whose evaluators were read directly, with $k = 4$ paired seeds per context and an LLM-agent audit of every count. The original answers on 55/76 false and 75/80 true completions. The merge answers on 47/76 and 66/80: false answers change by -10.5 points (95% CI -25.0 to 0.0), true ones by -11.3 (-22.5 to -2.5), a selectivity (first difference minus second) of +0.7 (-16.1 to +16.3). On feasible tasks, the merge is a lever without a discriminator. The closed-form edit answers on 42/76 and 68/80: against the original, false answers change by -17.1 points (-28.9 to -6.6), on 9 of the 19 tasks rather than one context, and true ones by -8.8 (-18.8 to 0.0), a selectivity of -8.4 (-22.6 to +5.9). Its point estimate favours false completions, but the interval includes zero: neither a selective repair nor its absence is established. The drop survives removing the three tasks that overlap its construction or preservation sets (-16.7 points on 18 false completions, -29.2 to -5.6). Its random control, a single draw that left the screening contexts in place, here raises false answers by +15.8 points (+5.3 to +27.6) and leaves true ones unchanged (0.0); the learned edit lies -32.9 points below it on false completions (-47.4 to -19.7), but because the control itself moves the result, the contrast with the original above is the relevant one. Two other perturbations that do not target the decision also raise false answers on this cohort. Setting to zero the four attention heads ranked most selective in the attribution of §F.2 changes them by +21.1 points (+9.2 to +35.5; healthy valid actions -3.1; heads selected on this same cohort, a screen not yet reviewed), and adding the full task vector of TMax-4B, an RL-trained Qwen3.5-4B and the only one of four planned RL arms that was run, by +7.9 (+2.6 to +14.5; healthy valid actions -4.7, failing the healthy gate). All intervals are task bootstraps with 100,000 resamples, uncorrected for the four contrasts.

F.6 Online evaluation

No candidate passed the gate, so none was compiled into the weights (Appendix O states the compilation step, Proposition 1); five arms were run online, each paired by task with an original run on the same harness, and the arms served by Modal with an original also served by Modal, on the same GPU type, image and vLLM version but from a separate account (Table 3). The `answer` output-row edit changes only the row of the (untied) output head for the first token of `answer` (2,560 weights), lowering its logit by about 40 at the tool-name positions where it was calibrated; the corrective

prompt adds one line to the end of the system prompt: "Before calling answer, look at the current screenshot and check that every requirement of the task is visibly satisfied; if anything is missing or unconfirmed, continue working instead of answering." A sixth arm, the output head scaled by 1.6 (an effective sampling temperature of 0.5), was started on the reserve and the TEST split, but its server froze from 00:44 to 03:22 UTC, so both runs were invalidated and are not reported. Two reserve tasks score 0 under the H200-served original and 1 under both Modal-served arms; they score 0 under the Modal-served original as well, so the gap came from run-to-run variation, not from the server. The harnesses do differ: two `os` tasks score 0 on the local harness and 1 on the GitHub one, so runs from different harnesses are never pooled. The merge also produced 6 unparsable steps against none for the H200-served original, a sign that it degrades the output format. The decision rule for the merge was fixed before the comparison with the Modal-served original was read, but after the merge's own reserve scores had been seen, so only its TEST part is strictly prospective: the merge and an original served by Modal are compared on the 45 reserve and 120 TEST tasks, and a gain is claimed only if the lower bound of the task-bootstrap 95% CI of the paired difference is above zero. By this rule, the merge shows no detectable gain: over the 165 paired tasks it scores 92.5/165 against 89.3/165 (Δ +1.9 points, 95% CI -4.3 to +8.0; 18 against 14 tasks better and worse), and on the TEST split alone, the strictly prospective part, 70.5/120 against 70.3/120 (Δ +0.2, CI -6.5 to +7.5). On the same TEST tasks the H200-served original scores 72.1/120, +1.5 points above the Modal-served one (CI -4.5 to +7.6): the noise between two servings of the same weights is as large as the merge's effect. ExPO was also run on the TEST split without having passed the gate, a decision of the LLM agent coordinating the study, reported as such: 70.3/120 against 72.1/120 (Δ -1.5 points, 95% CI -8.9 to +5.7; 11 against 12 tasks better and worse). All online scores are the official OSWorld score, a mean over tasks with partial credit included. With few tasks scoring differently between two runs (for the merge on the reserve, 7 against 4 better and worse), the bootstrap bounds move by up to a few points with the resampling seed; we report seed 0 throughout, and an LLM-agent recomputation agrees on every point estimate. Counting only full successes, the TEST difference for ExPO is -1.7 points (95% CI -9.2 to +5.8). A paired difference smaller than about 17 points would go undetected with 80% power for ExPO on the reserve; the same computation gives about 21 and 23 points for the merge and the closed-form edit on the reserve, 50 for the output-row edit, 10 for each TEST comparison and 9 for the 165-task primary comparison, so these comparisons exclude only large effects. A cheaper paired design, branching from recorded episodes, was piloted: replaying a recorded episode up to its last step reproduces the screenshots in 9 of 10 validation episodes, and from the final step of 5 recorded episodes (two continuations each, both arms served by Modal) the original succeeds in 5/10 continuations and the merge in 4/10, with one episode whose replay diverges from its first step. The only failed episode that an original continuation recovers, `gimp/fbb548ca`, is an officially infeasible task, where the continuation states that the requested theme does not exist; on the feasible false successes whose replay is faithful, no continuation succeeds (original 0/2, merge 0/2). The pilot shows no advantage for the merge, no repair of a feasible false success, and is too small to show anything else.

G Feci Guard: an external verifier

If the model does not separate a false "done" from a true one, the check has to come from outside, and it should judge the final state rather than the agent's account of it, as τ-bench does by scoring the final database state, alongside the information the agent gives in its answers [47]. Feci Guard is the simplest form of that check. When the agent calls `answer`, unless the answer declares the task infeasible, the guard takes a fresh screenshot and asks a separate verifier model to list every requirement of the task, to mark each one as met, unmet or not visible with its evidence, and to return a verdict. An INCOMPLETE verdict, which requires at least one requirement visibly unmet, refuses the answer and returns to the agent, as the answer's tool output, the missing requirement and a next action; the episode continues. A COMPLETE verdict, a timeout or an invalid output lets the answer through. At most two answers are refused per episode, and the third passes unchecked. A refusal always says what is missing, because blocking a false "done" without an explanation repaired none of the feasible false successes of the branching pilot (§F.6). The verifier (Claude Sonnet through the Claude Code CLI alias `sonnet`, whose exact served version was not recorded, a fixed prompt and JSON schema, a 55 s timeout) and the decision rule were frozen and registered, in a self-declared registration in our repository, before any evaluation.

Judge	false successes detected	true successes rejected	gap, points [95% CI]	no verdict
Holo-3.1-4B judging its own episodes	35/66 (53%)	97/202 (48%)	+5 [−13, +23]	6
Holo4-27B, same prompt	31/66 (47%)	63/202 (31%)	+16 [−3, +33]	5
Holo4-27B, Feci Guard prompt, raw verdict	31/66 (47%)	49/202 (24%)	+23 [+5, +40]	23
Holo4-27B, Feci Guard prompt, guard's rule	27/66 (41%)	38/202 (19%)	+22 [+7, +38]	23
Claude Sonnet, frozen Feci Guard verifier	45/66 (68%)	21/202 (10%)	+58 [+41, +74]	0

Table 4: An external verifier separates false from true successes; the agent judging itself does not. The same 268 completed episodes of the original agent (66 feasible false successes, 202 true successes, 90 tasks), each judged once from the final screenshot only (the Holo models at temperature 0, the Claude verifier with the CLI's default settings). Detection is an INCOMPLETE verdict on a false success; rejection, one on a true success; a call with no verdict lets the answer pass and is counted in the last column. Gap: detection minus rejection, with a percentile bootstrap over tasks. The last three rows use the frozen Feci Guard prompt (a JSON verdict per requirement, INCOMPLETE only if one is visibly unmet); the first two use a free-form prompt. "Raw verdict" counts every INCOMPLETE; "guard's rule" counts only those that mark a requirement unmet, as the guard does before refusing (15 of Holo4-27B's 80 do not; all of Claude's do, so its row is the same under both readings). An LLM-agent recomputation reproduces every count of the raw rows.

Offline, the verifier separates false from true successes. On the same 268 episodes of the original agent, judged from the final screenshot only, the frozen verifier detects 68% of the false successes and rejects 10% of the true ones, a gap of +58 points (95% CI +41 to +74, bootstrap over 90 tasks; Table 4). Holo-3.1-4B judging the same episodes detects 53% and rejects 48% (gap +5, −13 to +23), not distinguishable from chance, and the paired difference between the two judges is +53 points (+26 to +79). A larger model of the same family, Holo4-27B with the same prompt as Holo-3.1-4B, rejects fewer true successes but detects no more false ones (gap +16, −3 to +33). With the verifier's own prompt, Holo4-27B returns INCOMPLETE on 47% of false and 24% of true successes (gap +23, +5 to +40; 23 calls without a verdict, counted as passes): +18 points above Holo-3.1-4B (−5 to +40) and −35 below Claude (−59 to −10), on raw verdicts. But 15 of its 80 INCOMPLETE verdicts mark no requirement as unmet, and the guard's rule lets those answers through; under that rule it detects 27/66 (41%) and rejects 38/202 (19%), a gap of +22 (+7 to +38). Claude's counts are the same under both readings: each of its INCOMPLETE verdicts marks a requirement unmet. At equal prompt, Claude's advantage comes mostly from the model; the prompt helps Holo4-27B by lowering its false rejections from 31% to 24%, partly because its 15 calls without a verdict on true successes count as passes (26% on valid verdicts only). What a refusal is worth depends on the base rate. Among the tasks of the TEST split that the original, served by Modal, closes with `answer`, 27 score 0 and 65 score 1 (5 partial scores left out), a false share $f \approx 0.29$. With the verifier's detection d and false rejection r above, the precision of a refusal, $df/(df + r(1 - f))$, is about 0.73: roughly one refusal in four would hit a true success. This is an estimate, since the rates come from neighbouring distributions, not the same one. This is the measured basis for an external verifier, with three limits. One screenshot cannot show a saved file or a hidden setting, so detection cannot reach 100%. Each judge ran once. And Claude may have seen public OSWorld tasks.

A stronger judge. Claude Opus, given the two screens before the answer and the list of actions, detects more false successes than the frozen verifier on a held-out half, 20/29 against 16/29 (+13.8 points, paired task-bootstrap 95% CI +3 to +30; few discordant pairs, so the interval is approximate), with false rejections that rise by +4.3 points (−5 to +15). The variant was selected on a development half and confirmed once on a held-out half drawn by task hash and committed before any test: on that half it detects 69% (20/29) and rejects 12.9% (15/116), against 55% (16/29) and 8.6% (10/116) for the frozen verifier, with a paired bootstrap over its 47 tasks. A strict prompt demanding file evidence was rejected on the development half: it detected 35/37 false successes but rejected 33/86 true ones. The protocol fixed a target of at least 80% detection with at most 10% false rejection on the held-out half; it is not met. The split was committed before the development runs, but the protocol text was first committed together with the development results, and the Opus model is the unpinned CLI alias `opus`. The stronger judge has not been evaluated online.

Online, the false "done" rate fell, mostly in episodes the guard never saw. The guarded agent was run once on the 120 tasks of the TEST split, under a rule and an analysis script committed before its results were read, and paired by task with the original served by the same Modal deployment on the same runners. Its false "done" rate, a final `answer` on a task scored 0, fell from 22.5% to 10.8% (−11.7 points, 95% CI −19.2 to −4.2), which formally meets the registered primary criterion (an

			false "done"				50 steps reached	
Guarded episode	n	refusals	original	guarded	B (0→1)	C (1→0)	original	guarded
agent never answered (guard never ran)	28	0	9	0	1	1	17	28
answered, guard never refused	75	0	12	10	5	1	2	0
guard refused at least once	17	22	6	3	2	2	5	10
answered at least once (rows 2 and 3)	92	22	18	13	7	3	7	10
all tasks	120	22	27	13	8	4	24	38

Table 5: About two thirds of the online fall in false "done" come from episodes in which the guard never ran. The guarded run and the original served by the same Modal deployment, paired over the 120 TEST tasks, split by what happened in the guarded episode. False "done": final `answer` on a task scored 0. B and C as registered (score below 1 to 1, and 1 to below 1). Last two columns: episodes that used all 50 steps.

upper 95% bound below zero). Its OSWorld score went from 58.6 to 60.2 (+1.6 points, −4.4 to +7.7), which does not meet the secondary criterion (a lower bound above zero): we claim no score gain. 8 tasks went from 0 to 1 and 4 from 1 to 0 (served twice, the original flips 9 tasks from 1 to 0 and 5 from 0 to 1), so the registered usefulness rule (at least as many tasks gained as lost, and fewer false "done") is formally met too. Against the original served by the pod, the false "done" rate changes by −13.3 points (−20.8 to −5.8) and the score by +0.1 (−6.5 to +6.6).

Table 5 splits these counts by what the guard could do on each task. In 28 of the 120 guarded episodes the agent never called `answer`, so the guard never ran; the original had 9 false "done" on these tasks, so 9 of the net drop of 14 (27 to 13) involve no guard action. Never answering is within the run-to-run variation of the original: the guarded agent never answered on 28 tasks, the original on 23 (Modal) and 20 (pod); the guarded arm and the Modal original disagree on 11 against 6 tasks (exact McNemar $p = 0.33$), the two servings of the same original on 5 against 8. The guarded agent loop is the original loop up to the first `answer` (same 50-step budget, sampling at $T = 0.8$, unseeded, same prompt and checkpoint); a refusal only consumes a step and continues the episode. The 38 guarded episodes that used all 50 steps are the 28 that never answered and 10 that were refused and then ran out of steps. On the 92 tasks where the guarded agent answered at least once, and the guard could act, false "done" goes from 18 to 13; this part was not registered as a separate test and we do not test it. The direct evidence of the guard is the 17 tasks it refused on (22 refusals), where false "done" goes from 6 to 3, 2 tasks go from 0 to 1 and 2 from 1 to 0. So about two thirds of the drop come from episodes in which the guard never ran, consistent with the sampling noise of an unseeded agent, and we do not claim that the guard lowers the false "done" rate online. A seeded replication, or the same guarded loop with a neutral verifier that always returns COMPLETE, would settle it; neither has been run.

The registration was amended three times, each amendment committed before any comparison: a benefit and cost rule, before any full arm ran; after the pod serving the agent failed, and before any guarded episode, a move of the agent and the primary comparator to the Modal endpoint; and a single re-run of six tasks lost to an infrastructure failure, which also registered a secondary analysis on the 114 tasks available at that time that we do not report. Seven shards that failed at setup were re-run with identical settings without an amendment. The registration also planned a replication on the confirmation reserve and an online run of the escalation to Holo4-27B; neither was run. Like all our registrations, it is self-declared: timestamped commits in our own repository.

Replayed repairs. In one end-to-end episode on a LibreOffice Impress task the original had failed with a false success, the guard refused the first "done" because the strike-through stopped mid-word, the agent followed the feedback and fixed the line, and the second "done" passed; the official checker still scored the task 0, because the remaining difference lies in the saved file, which the screen does not show. The guard also runs in our side-by-side live demonstration of the original and the guarded agent. A repair measure replays 12 recorded feasible false successes of the original up to their final answer and continues each twice per arm, with and without the guard, for at most 15 new steps, on the same server at the same time, scored by the official evaluator. Its protocol section was written before the arms were launched but first committed when about six branches had already finished, so the registration is self-declared; the exclusion rule depends only on the replay prefix, which runs before the continuation. 9 episodes replay faithfully in all four branches (3 excluded), 18 branches per arm. With the guard 5 of 18 branches are repaired, without it 2 of 18. The paired

difference per episode is +16.7 points (95% CI 0 to +44, bootstrap over episodes; with nine discrete differences, the upper bound varies between +39 and +44 with the bootstrap seed); 2 episodes are better with the guard and 0 worse, a sign test gives $p = 0.5$, and over all 12 episodes the difference is +12.5 (0 to +33): the effect cannot be told apart from zero. The guard refused 12 branches (16 refusals in all), and 3 of them ended repaired, 25% (Wilson 95% CI 9–53%), from two episodes: on `calc/1de60575`, 2/2 against 0/2 without the guard, the verifier saw that the pivot table sat on a sheet with the wrong name and the agent renamed it; on `writer/0b17a146`, 1/2. `impress/8979838c` succeeds in all four of its branches as soon as the agent answers: its recorded false success does not reproduce under replay, so it accounts for both repairs without the guard and two with it, and only 3 repairs are specific to the guard. The costs are visible too: 7 refused branches reach the step limit without answering again, 2 pass at the refusal cap while still wrong, and on 2 episodes the guard accepts a wrong answer as complete in every branch. What is measured is a refusal plus coaching, since the feedback contains the verifier's suggested next action, not refusal alone. The refusal sometimes helps the agent repair, without statistical evidence at this stage.

Escalating to a stronger agent. The same twelve recordings were replayed with a different policy: when the guard refuses the original answer, the episode is handed, with the answer and the verifier's feedback in its history, to a stronger agent, Holo4-27B, for at most 20 new steps; the control hands every episode to Holo4-27B at the same point, without the guard. This contrasts the whole gated handoff with the stronger agent alone, not feedback alone. The protocol was first committed together with partial results (4 of 48 branches), and a clarification of the analysis, crediting a repair to the prediction only after a new action, was added during collection; the registration is self-declared. On the 9 episodes whose replay is valid in all four branches, the official score is 1 in 11/18 branches with the guard against 2/18 without it (9 against 0 after a new action), a paired difference of +50 points (95% CI +22 to +78, bootstrap over episodes; 5 episodes better, 0 worse). The result holds with a stricter replay check (8 episodes, +50, +13 to +88) and with all 12 episodes (+33, +4 to +63). After an actual refusal, 9/11 branches are repaired after a new action. This is a replay measurement: escalation to Holo4-27B was not run online, and the change it predicts on the TEST split, +6 to +12 points depending on the unmeasured cost of a wrongly refused true success, is a prediction.

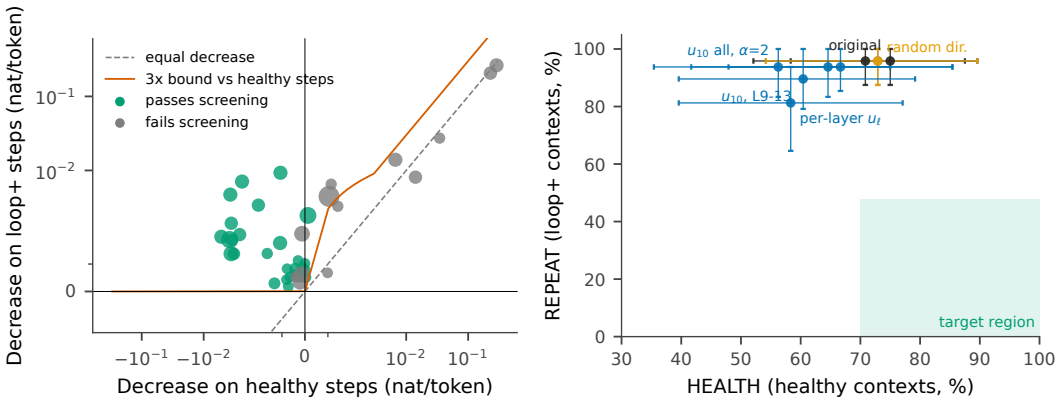
H A first negative result: loops in Holo-3.1-0.8B

We first tried the recipe on the defect behind the weight-space programming result, looping, in Holo-3.1-0.8B on MiniWoB++. On 172 rollouts over 24 tasks the model succeeds on 40.7%; loops affect 24.5% of failures (106 LOOP+ steps), just below the 25% GO threshold. The GO was given on a partial diagnostic (92 rollouts, loops 26% of failures), and the full diagnostic's stored share (25.5%) disagrees with its own counts.

Detection works. At layer 10, the loop direction reaches AUROC 0.893 (Fig. 2a). The position baselines reach 0.70 (step index) and 0.72 (context length), and an independent refit of the direction reaches 0.904.

The weight edit is not selective. Two rounds screened 40 weight edits along this direction (layers, writers, top-k columns, α from 0.3 to 16; Appendix Table 15). The screening winner was frozen before the reserved seeds were read. It edits 1,953,863 weights (0.229% of the model). Its screening selectivity did not carry over beyond the 40 screening contexts: on the full development seeds it already lowers healthy actions more than loop actions (Table 12). On reserved seeds it lowers the log-probability of loop actions by 2.31 mnat/token and that of healthy actions by 8.92 (ratio 0.26, against a target of 3). A random direction under the same scope gives a similar ratio (0.31), and neither lowers loop actions significantly: both 95% task-bootstrap intervals include zero (Table 12, Fig. 9a).

Steering does not reduce loops. In free generation, the original model repeats on 95.8% of loop samples and matches the healthy action on 75.0%. The pre-registered target was REPEAT $\leq 47.9\%$ with HEALTH $\geq 70.0\%$. The best arm, projecting each layer's own direction at all layers, reaches REPEAT 81.2% at the cost of HEALTH 58.3% (Fig. 9b, Table 13). The random direction (95.8%/72.9%) cannot be told apart from the identity hook (95.8%/72.9%). The loop direction is a marker, not a usable lever.



(a) Offline screening of 40 weight edits.

(b) Behavioural steering arms.

Figure 9: On the 0.8B, edits large enough to matter hit healthy actions as hard as loops. (a) Teacher-forced decrease in log-probability of the recorded action, loop versus healthy steps, on 40 fixed development contexts (symlog axes; marker area grows with the number of weights edited). The red line is the screening bound against healthy steps (loop decrease at least 3 times the healthy decrease); the criterion applies the same bound to loop− and successful steps, so 6 failing candidates lie above the line. Every passing candidate lies above it, with loop decreases of at most 0.009 nat/token. (b) Loop rate versus healthy-action rate per steering arm, 95% bootstrap CIs over contexts [8]. The shaded region is the pre-registered target, and no arm enters it.

I Discussion: why detecting is not causing here

A marker of a state is not the cause of an action. Looping is a property of the trajectory: the agent repeats because the screen did not change or the target was misread. A direction that encodes "this episode is stuck" can be read reliably while the repeated action is computed from features that do not depend on it. The global-erasure control shows this directly. Probes in computer-use agents can reach a very high AUC that is not, by itself, evidence that they detect the intended property [22], and format-confounded probes can lack any functional role [32]. Our 0.8B result is a case where the probe does track the intended property and is still a marker, not a usable lever.

False success looks like a belief that the task is done, acted on without checking. The 4B evidence points one way. The model says "done" on most false completions as on true ones, and its reasoning writes the conclusion first in both cases, a property of its output format (§C.1). The tool choice becomes visible late, at the tool name, after a thought that already declares success, through a circuit that reads the tool definition and treats true and false completions the same (§F.1). Pushing toward low value makes the model write that it should check, without making it check. This matches reports that stated confidence tracks commitment more than correctness [18] and that the decision to stop thinking is localised in a few late MLPs [7]. The perceived-completion direction (§F.4) adds that, on 14 matched pairs, its projection reads false completions as complete. A public report on the hosted Holo3.1 35B, whose screen capture failed on the last three steps of a run while the agent kept acting and then reported success [6], is an extreme case of the same pattern in a larger model of the family: the claim of completion is not conditioned on evidence that the task was done. Whether that state is a misperception of the screen or a commitment to finish, removing it is likely to suppress true completions too, and the FS− criterion of our gate exists to catch exactly that. Two samples from modified models, not from the original, suggest that at least some false "done" are not reading errors. On a Chrome rental task, under the arm that adds the "verify" versus "conclude" direction at one times the mean residual norm (§F.3), the model reads the current date "Nov 8" on screen, then states that "Nov 10–11" is next month and answers "done"; a Gaussian control direction at twice that norm produces the same contradiction, so the arm does not cause it. A second sample on the same task, under the "Wait" edit, goes further: it derives that next month is December, says the dates must be changed, then decides that the main requirement (sorting by seats) is met and answers "done". Two outputs of modified models are not a population claim about the original, but they suggest that a false "done" can be the partial satisfaction of a conjunctive task, with the unmet requirement noticed and then dropped. A repair may therefore have to enforce that every requirement holds, rather than act on looking or on the decision to stop.

Does the agent know? An external reader asked whether an agent that knows a requirement is still unmet answers "done" anyway. Under a protocol registered before annotation, a blind annotator (Claude Sonnet through the `claude` CLI) read the instruction and the note and thought of the last three steps of every episode in which Holo-3.1-4B ends with `answer` on the TEST split, in both servings of the original (agreement 20/20 with a second blind Sonnet annotation of 20 random rows, 17 of them "no"; a first re-annotation whose brief also counted still-to-do steps such as saving agreed on 10/20). When it ends with `answer`, the agent rarely writes in those steps that a requirement remains unmet: in 9/56 false successes (16%) against 10/133 true successes (8%), a difference of +8.6 points (95% CI −0.8 to +20.8; one-sided Fisher $p = 0.07$). Most of these mentions state that the task cannot be done, and 5 of the nine on the false-success side are correct infeasibility answers that our harness scored as "done". Dropping a known constraint therefore explains few false successes (two or three of the 56), and a free rule based on this signal would catch only 16% of them, against 68% for the external verifier (on a different set of episodes, so the comparison is indicative).

Implications for agent developers. We found no direction whose removal lowers false "done" selectively, in screens able to detect only large effects; within our probes, nothing separated false from true completions beyond the episode's length except the hesitation between samples (Table 1, §F.5), and no edit that moved the decision showed an established selectivity on feasible tasks: the merge showed none, and the closed-form edit's interval includes zero. Our results suggest that a fix must bring in information that we could not find in the model's activations or text when it stops: an external verifier that checks each requirement of the task against the screen before `answer` is accepted (§G: offline, it separates false from true successes where the agent, judging itself with a free-form prompt, does not; online, the guarded run had fewer false "done" with no measurable score change, but most of that drop came from episodes in which the guard never ran, so it is not attributed to the guard), or training that targets false successes, for instance with rewards that penalise episodes ending in a false "done". Weight surgery of the kind tested here has not shown that it can supply it.

What would change our mind. The official analysis has not met the first condition we set, a direction on at least 40 FS+ steps that beats position: its best direction is within noise of the position baselines (Table 1). What remains is an intervention that moves FS+ answers while sparing FS− on unseen tasks, perhaps at the larger doses the ceiling effect calls for, and a paired online gain on the confirmation reserve larger than the 17 points our reserve can detect. Measuring how many false-success states can still be repaired when `answer` is forbidden would bound what any such edit can gain; we have not measured it.

J Related work

Directions, steering and ablation. Behaviour-specific directions, built from contrasts of activations (differences of means, a single contrast pair, or principal components), are added at inference [21, 30, 39, 53] or projected out [2]; linear structure has also been reported for truthfulness [24]. Refusal in chat models is mediated by a single direction, and ablating it everywhere equals orthogonalising every writer [2], the equivalence of §O. Venhoff et al. steer backtracking and uncertainty in thinking models [40]. Weight-space programming removes a looping habit by editing `down_proj` columns [37].

Agents, tools and termination. Tool identity is linearly readable and steerable in single-turn settings, with steering inside agent loops left untested [43]. A direction can set the rate of tool calls [4], agents encode when to call a tool before reasoning [36], and steering has served as procedural memory for agents [50]. Huang et al. study when GUI agents should not act [14]: their CONFLICT-GUARD checks, before acting, whether an instruction conflicts with itself or with the screen, and steers the agent toward termination; Feci Guard checks the opposite end, whether a completion the agent has already claimed holds on the screen. Models carry a value axis for being on the right track [16]; confidence tracks commitment [18] and supports self-detection of errors [19]; overconfidence has a mechanistic account [51]; and the decision to stop thinking is localised in a few late MLPs [7]. On the evaluation side, τ-bench judges an agent by the final state of the environment, together with the information its answers must contain, rather than by its account of the task, and measures reliability over repeated trials with pass^k [47]; Feci Guard (§G) applies the first idea at run time, on the screen. A model-based evaluator of digital-agent trajectories can serve both as a judge and as

a signal for refinement [29], whereas a model asked to correct its own reasoning without external feedback often does not improve and can get worse [13]. Holo-3.1 judging its own final screen (§G) is closer to the second setting, and the external verifier to the first.

Pitfalls. A probe shows decodability, not use [3]; control tasks [12] and confound residualisation [32] are needed. Steering vectors generalise unevenly [34, 38], can compromise safety [17], push activations off the natural manifold [27], and are often beaten by prompting [44]. Patching has its own best practices [11, 49] and is the main tool of circuit discovery [41], and locate-then-edit weight editing builds on causal tracing [25], scaled to many edits at once by MEMIT [26]; merging fine-tuning deltas is task arithmetic [15].

K Limitations

- **Small causal samples.** Each 4B screen uses six contexts from five tasks, and the reasoning analysis uses four tasks. These are screens for a lever, not estimates of effect size. A small effect on other tasks could have been missed, and we report "no admissible effect found", not "no effect exists".
- **The metric targets the wrong action on infeasible tasks.** 9 of 29 false successes are claims of completion on infeasible tasks, where `answer` with a refusal is the correct tool; one such context enters several screens and the ceiling test. Appendix M gives every count with and without it.
- **Many uncorrected comparisons.** Every arm of Tables 2, 7 and 8, the merges and the closed-form edits was screened on the same few contexts, and five arms (8 non-original online runs) were run online, without correction for multiple comparisons. No intervention is claimed to be a selective repair; corrected p-values were not computed.
- **The online drop in false "done" is not shown to be caused by the guard** (§G). In 28 of the 120 guarded episodes the agent never called `answer`, so the guard never ran; 9 of the net drop of 14 false "done" (27 to 13 tasks) are on those tasks. Where the guarded agent answered at least once (92 tasks), false "done" goes from 18 to 13. How often the agent never answers varies between runs within noise (28 guarded, 23 for the original served by Modal, 20 for the original on our H200 server; sampling at $T = 0.8$ is unseeded). The guarded loop is identical to the original up to the first answer, except that a refusal costs a step and the episode continues. A seeded replication, or a control with a neutral verifier that always accepts, was not run.
- **Escalation and replication not measured online.** Escalation to Holo4-27B is measured in replay only, on 9 episodes; the guard was not replicated on the reserve, and escalation was not run online. The guard's cost and latency were measured offline only: $7.60 for the 268 verifier calls of the judge comparison (about $0.03 per check, as reported by the Claude CLI) with a median latency of 6 s; the online run's cost was not computed.
- **The test split was used by 5 valid model runs and the guarded agent** (§F.6, §G): the original (served twice), ExPO, which had not passed the gate, a partial corrective-prompt run, and the CUA-Lite merge under a decision rule fixed after its reserve scores had been seen. A further arm, the output head scaled by 1.6 (sampling at an effective $T = 0.5$), was started on the TEST split and on the reserve; its server froze from 00:44 to 03:22 UTC, so both runs were invalidated and are not reported.
- **The confirmation reserve was used outside the protocol** (§F.6). The protocol kept it to confirm a committed candidate; no candidate was admitted, and five non-admitted arms were run on it: ExPO, the `answer` output-row edit, the corrective prompt, the CUA-Lite merge and the closed-form edit. It is no longer an untouched set for a future candidate.
- **Self-registered rules, LLM auditors and an LLM coordinator.** Our registrations are timestamped entries in our own repository, not a public registry; the primary online rule was fixed after the merge's reserve scores had been read, so only its TEST part is strictly prospective. The semantic audits and reviews called independent in this paper are LLM-agent audits, not validated against human raters, and some protocol decisions (for instance ExPO on the TEST split) were taken by the LLM agent that coordinated the study.
- **One model, one benchmark.** Every result concerns Holo-3.1-4B (and the 0.8B) on OSWorld (and MiniWoB++) with one harness; the mechanistic analyses are exploratory, select and estimate on the same few contexts, and use artificial teacher-forced inputs. The Claude verifier may have seen public OSWorld tasks, and the training data of the CUA-Lite donor may overlap OSWorld tasks (§F.5). The judge was called through the Claude CLI alias `sonnet` (Claude Sonnet,

version not pinned); the Holo4-27B weights are `Hcompany/Holo4-27B` from Hugging Face at revision `4de03cfe`, as recorded in our deployment manifests.

- **Exploratory direction analysis.** The official analysis passed the 40-positive gate, but the second pass stopped at 80 tasks, so it is labelled exploratory; the steering screens predate it.
- **Automatic labels.** The 1% screen-change threshold, the 15 px click radius (two clicks, scrolls or drags less than 15 px apart, after rescaling to screen pixels, count as the same action when labelling repetition loops) and the refusal regex are design choices, fixed before analysis but not validated against human labels.
- **Harness.** 50 steps, three screenshots and a third-party public harness (RealtimeGUI), not H Company's, whose infeasibility rule is broader than the official OSWorld adapter's, whose system prompt already asks the agent to verify before answering, and whose OSWorld copy changes the page-load wait of the Chrome evaluator getters (§B); absolute scores are not comparable with published numbers, paired comparisons are. In both servings of the original on the TEST split, 5 answers are correct statements of infeasibility that the harness's regex (`run_holo.py`, line 67) misses: the harness then sends `DONE`, the task scores 0, and the answer counts as a false "done". We kept the official score. Two of them (`chrome/ae78f875` and `os/b3d4a89c`) are among the 27 false "done" of the original served by Modal against which the guard is compared (§G); `chrome/ae78f875` scores 1 in the guarded run only because the regex recognised the reworded answer there, not because of a guard refusal.
- **One family of directions.** Difference-of-means, matched pairs, PCA, attribution-selected neurons and heads. Sparse autoencoders and multi-direction subspaces were not tested.
- **Random controls are not neutral.** On the strictly feasible cohort the random control of the closed-form edit itself raises false answers by +15.8 points (+5.3 to +27.6), and it is a single random draw, so a contrast with it overstates how specific the learned edit is (§F.5).
- **Shared compute.** Research jobs shared one GPU with the rollout server and paused on a latency flag; interrupted runs are disclosed and resumed exactly, never counted as results.

L Reproducibility and how this work was done

Models: `Hcompany/Holo-3.1-0.8B` at revision `72da4c53` and `Hcompany/Holo-3.1-4B` at `8c88265a`. The companion repository is a snapshot. Its full development history is in https://github.com/martinbon39/ComputerUseHackaton, archived read-only. Its git history records when each pre-registration, protocol and amendment, and each analysis script, was committed (git times are set by the committer, not by a third party), and every hash of the project's own history cited in this paper or the companion repository refers to it (`docs/PROVENANCE.md` lists them). Every macro, table and figure in this paper is produced by `paper/figs/*.py` from result files copied into `paper/data/`. Each macro in `paper/generated/num_*.tex` carries a comment naming its source, and `build.sh` regenerates the macros, tables, figures and PDF. Numbers typed directly in the prose are not generated. The data copies are not complete. For the online OSWorld arms, `data/` holds the per-task scores and metadata of every arm (and the last steps of three TEST runs), not the trajectories or screenshots; those stay on our server, outside the repository. The 268 final screens of the judge comparison are in the repository (`experiments/verif_study/images/`). For the pod experiments, `data/pod_astra/` holds the reports and summary files copied back from the pod. Per-generation outputs are included for some arms (the residual-patching lots, the CUA merge, the anchoring test, one value-steering lot) and absent for others: the power study, the RL task-vector addition, the note-to-tool attribution, and the folders `mit2`, `short`, `ratio`, `regmean`, `tact`, `neurons` and `stuck`, whose numbers rest on those summaries alone. The experiments were planned, run and reviewed by AI agents (Claude Code sessions, and GPT-6 Astra through the Codex CLI) under human supervision; Appendix S describes the organisation, the safeguards and the rules that were not respected. Many results were checked by a fresh-context agent that had not produced them, but not every result was.

M Infeasible contexts: every count with and without them

Infeasible tasks are not rare among our labels: 11 of the 47 FS+ steps of the two diagnostic passes come from infeasible tasks. One FS+ context, Chrome `480bcfea` at step 4, comes from an infeasible task: "I do not like the design of the new 2023 chrome UI. I want to keep using the original one. Can you disable the new 2023 version chrome UI for me?" The 2023 interface cannot be switched off,

so the correct final action is `answer` with a statement of infeasibility. The model instead claims that it disabled the new interface, which is a false success, but a non-`answer` action on this context is not a repair, and $P(\texttt{answer})$ is the wrong metric there. The context enters most screens of Tables 2 and 8, the merges and the first context of the ceiling test. Table 6 gives every affected count with and without it. Four readings change. The three non-`answer` calls of H6 are all on this context, so on the feasible FS+ context H6 answers 8/8 like the original. The only `answer` calls of H3 at $\lambda = 1$ are on this context too (0/8 without it). The FS+ non-`answer` calls of H10 at $s = 0$ also fall on this context (8/16 answers overall, 8/8 on the feasible context). And without it, the CUA-Lite [28] merge at $\beta = 0.5$ answers on 3/8 feasible false completions, a larger drop than in §F.5, still with true completions at 2/16. A second infeasible task, Writer `bb8ccc78`, is the other FS+ context of the perceived-completion screens (§F.4), and there the model's answer states that the task cannot be done, in words the harness rule misses: both FS+ contexts of those screens are infeasible, so the perceived-completion rows of Table 2, including their "excl." column, contain no feasible false completion. None of the other conclusions changes: the arms that leave FS+ at its original rate do so on both contexts.

Source	Arm	fs+ `answer` (all)	fs+ `answer` (feasible contexts only)
Steering screens (Table 2)	original reference	8/8	4/4*
	residual addition L16	8/8	4/4*
	residual addition L24	8/8	4/4
	"conclusion" direction, L31	8/8	4/4*
	Value Axis, value, L24	8/8	4/4
	Value Axis, commitment, L24	8/8	4/4
	Value Axis, value, L18	8/8	4/4
	Value Axis, commitment, L18	8/8	4/4*
	circuit C1, $\alpha = 0.5$	8/8	4/4
	circuit C1, $\alpha = 1.0$	8/8	4/4
	circuit, random control, $\alpha = 0.5$	8/8	4/4
	perceived completion, $\alpha = 0.5$	5/8	0/0
	perceived completion, $\alpha = 1$	4/8	0/0
	perceived completion, $\alpha = 2$	5/8	0/0
	perceived completion, $\alpha = 4$	5/8	0/0
	perceived completion, random control, $\alpha = 4$	7/8	0/0
	perceived completion, reference (original)	6/8	0/0
	verify, calibration baseline	4/4	0/0
	verify, dose 1	8/8	4/4
	verify, dose 2	8/8	4/4
	verify, random control, dose 1	8/8	4/4
	verify, random control, dose 2	8/8	4/4
	weight edits, vLLM reference	8/8	4/4
	"Wait" output row	6/8	3/4
	linear-attention retention $\times 0.8$	8/8	4/4
	linear-attention retention $\times 1.25$	8/8	4/4
	H4 audit: false-completion claims	5/8	2/4
Mechanism-level edits (Table 8)	original (re-served)	16/16	8/8
	H2	16/16	8/8
	H3, $\lambda = 0.5$	16/16	8/8
	H5, $\lambda = 0.5$	16/16	8/8
	H6	13/16	8/8
	H2, $\lambda = 2$	16/16	8/8
	H9	16/16	8/8
	H10, $s = 0.5$	16/16	8/8
	H10, $s = 0$	8/16	8/8
	H3, $\lambda = 1$	3/16	0/8
	audit: original, false-completion claims	16/16	8/8
	audit: H6, false-completion claims	13/16	8/8
	audit: H2, false-completion claims	16/16	8/8*
	audit: H3, $\lambda = 0.5$, false-completion claims	16/16	8/8*
	audit: H5, $\lambda = 0.5$, false-completion claims	16/16	8/8*
Donor merge	original (re-served)	16/16	8/8
	CUA donor, $\beta = 0.25$	15/16	7/8
	CUA donor, $\beta = 0.5$	11/16	3/8
	Fara donor, $\beta = 0.5$	16/16	8/8

Not included: `neurons` (task 480bcfea is a construction task, not evaluated); `ratio`, `mit2`, `short`, `tact` (no task id 480bcfea in their data files). * every fs+ output answered, so the split follows from the per-context sample count.

Table 6: FS+ `answer` counts over all FS+ contexts and over feasible ones only, for every arm that includes the infeasible context `480bcfea@4`. The feasible-only column also removes Writer `bb8ccc78`, the other infeasible context of the perceived-completion screens, which therefore have no feasible FS+ context (0/0). *Split implied exactly by the arm's cohort, because every FS+ output answered. Sources whose files carry no task identifiers are listed at the bottom.

Edit (no gradient, no data)	fs+ `answer`	fs− `answer`	invalid/truncated
Original (re-served)	16/16	14/16	0/32
ExPO $\lambda = 1.1$, language layers	16/16	15/16	0/32
ExPO $\lambda = 1.2$, language layers	16/16	13/16	0/32
ExPO $\lambda = 1.25$, language layers	16/16	13/16	0/32
ExPO $\lambda = 1.5$, language layers	16/16	13/16	1/32
ExPO $\lambda = 1.5$, attention only	16/16	15/16	0/32
ExPO $\lambda = 1.5$, MLP only	15/16	10/16	0/32
Holo $+ 0.5$ (Qwen3.5-4B $-$ Base)	16/16	16/16	0/32

Table 7: Direction-free weight edits leave false "done" unchanged. Free generation on two false-completion and two true-completion steps, 8 samples each, vLLM with the production harness call. Exploratory pilot.

N Direction-free and mechanism-level weight edits

This appendix details the weight edits summarised in §F.5: extrapolation of the fine-tune, and edits that target a mechanism rather than a direction. None moves the decision on a feasible false completion without moving true completions or breaking generation.

N.1 Edits without a direction: extrapolation and the Qwen post-training vector

Direction-free weight edits do not depend on locating anything. Holo-3.1-4B descends from the post-trained Qwen3.5-4B, not from its base: in the language layers, the fine-tune moves the weights by 4.7% of their norm, against 7.8% from the base and 6.4% between Qwen's base and post-trained models, and H Company's delta is almost orthogonal to Qwen's post-training (cosine -0.02). We therefore tested model extrapolation, $W(\lambda) = W_{\text{Qwen}} + \lambda\,(W_{\text{Holo}} - W_{\text{Qwen}})$ [52], on all language layers, attention only or MLPs only, and the addition of half of Qwen's post-training delta, in the spirit of reasoning vectors [48]. None changes false answers: they stay at 16/16 in every arm but one (15/16, MLPs only), where true completions drop more (Table 7). At $\lambda = 1.5$ one output in 32 loops to the token cap. The edit at $\lambda = 1.25$ keeps text and JSON intact, and it runs online as an arm of Table 3, because only OSWorld measures whether actions improve.

N.2 Mechanism-level edits change the text, not the tool call

Three more gradient-free edits target a mechanism rather than a direction. H2 resets the 48 memory-gate matrices of the 24 Gated DeltaNet layers [46] to Qwen's; H3 moves the 96 language MLP matrices halfway to Qwen's, leaving attention, memory, vision and the output head untouched; H5 moves the input embedding of `</think>` halfway toward the double-newline row, with the output head untied and kept. Each was served by vLLM on separate hardware against a re-served original, with the same 8 seeds on the same 6 development contexts. Each reproduces the original's tool counts exactly: 16/16 false and 15/16 true completions answer, every healthy action stays valid, no output is invalid (Table 8). Only 2/51, 0/51 and 4/51 outputs are token-identical to their paired original, and 21/51, 1/51 and 39/51 up to the end of reasoning (Fig. 10). We did not compare two runs of the unmodified model with the same seeds on a batched server, but H5 partly provides that reference: it changes only the input embedding of `</think>`, so its reasoning is computed by unchanged weights except where that token already appears in the prompt, and still 12 of its GUI reasoning prefixes differ from the original's (36/48 identical). About a quarter of reasoning prefixes may therefore differ through serving alone; H2 (20/48 identical) and H3 halfway (1/48) fall well below that level, so these two edits do reach the computation. What the table shows is that the tool call is robust to them.

One edit does move the tool call, and shows why a moved tool is not a correction. Scaling the attention scores of the eight full-attention layers by $s = 0.5$ (H6, a softer attention, 8 `q_norm` vectors) lowers false-completion answers to 13/16, while true completions answer 16/16 and healthy actions stay valid. All 3 non-`answer` calls fall on the infeasible context, where `answer` is the correct tool; on the feasible FS+ context H6 answers 8/8, like the original (Appendix M). A blind semantic audit, with original and edited outputs mixed, finds 0/16 useful actions, as for the original, and 13/16 outputs that still claim success (original 16/16); the 3 non-`answer` calls are unhelpful theme clicks

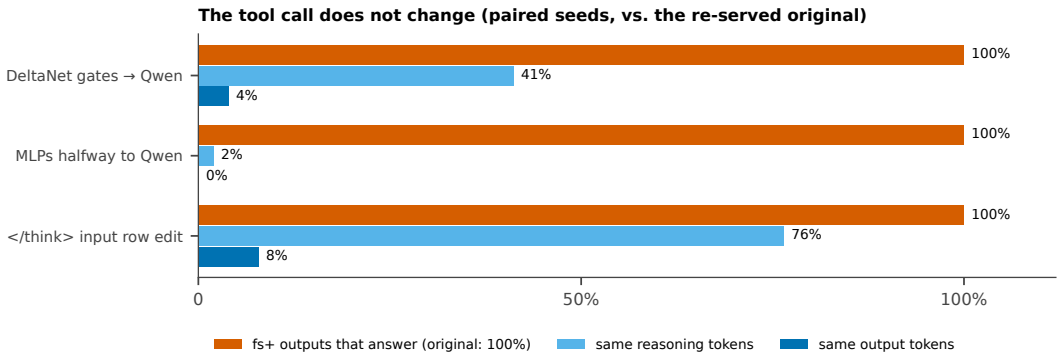


Figure 10: The tool call does not change under three mechanism-level edits. For three weight edits, share of false-completion outputs that still call `answer` (as the original does on all of them), and share of all 51 paired outputs whose reasoning, or whole output, is token-identical to the re-served original with the same seed. Six development contexts, 8 seeds each, plus three text prompts. Two runs of the unmodified model were not compared; H5, which leaves the reasoning weights unchanged unless `</think>` is in the prompt, gives a partial reference (36/48 identical GUI reasoning prefixes).

(GTK or Classic) justified by an incorrect prediction that they will restore the old interface, and none is useful. The tool moved; the task did not get closer to done. The second pre-registered dose, $s = 0.25$, was stopped at its pilot: the only output runs to the 3,072-token cap without closing its JSON, which closes the family. Four last edits close the series. Doubling the H2 gate edit ($\lambda = 2$, past Qwen), moving all 401 language tensors 20% of the way to Qwen at once (H9, a coordinated restoration rather than one block at a time), and halving the share of the query gain carried by the 64 rotary coordinates of each full-attention layer, the norm of the gain vector kept (H10, $s = 0.5$), leave the decision in place: false completions answer 16/16, 16/16 and 16/16, true completions 15/16, 16/16 and 15/16, and the blind audit finds no useful action. Removing that share entirely (H10, $s = 0$) repeats H6. False-completion answers fall to 8/16, all on the infeasible context, where 7 theme clicks and 1 wait replace an answer that should have explained the impossibility; the feasible context still answers 8/8, and 2 healthy steps end in truncated outputs, a loss of at least 12.5 points that rejects the edit.

The same picture comes from three other directions. Seven two-neuron conditional circuits (15,360 weights each), written into the MLP of layer 19 or 31 to fire on an imminent conclusion and push toward verification, give 35/35 false answers; the layer-31 version inserts one "Wait" in every output, after which the model sometimes recognises the wrong month on Budget and rationalises it. Even the input such a circuit aims for does not help: injecting a generic instruction to compare each requested detail with the visible evidence into the reasoning, with the original weights, late or early, gives 5/5 and 5/5 false answers. A narrow MLP edit adapted from the weight-space programming recipe [37] (64 columns in each of layers 17 and 20, 327,680 coefficients targeted) gives 8/8, 8/8, 8/8, 7/8 premature answers at $\alpha = 0.5$, 1, 1.5 and 6, and 0 useful actions in any arm; a random direction at $\alpha = 6$ gives 7/8, and true completions stay at 9–10/12 against 10/12. The last hypotheses closed the same way. Promoting the end-of-thinking token so that the model reasons less leaves `answer` rates unchanged at the dose that keeps outputs valid (false completions 13/20 against 13/20, true completions 18/20 against 18/20, on a 28-context panel), and breaks the JSON at higher doses (2/20 and 2/4 invalid outputs before the arms were stopped). In 3 further edits (penalising completion words, amplifying three task-reading heads, interpolating late MLPs toward Qwen), every dose gives 0/8 useful actions. An adaptation of TACT [35], which steers overthinking and overacting at the end of reasoning, could not be built as specified: after a blind audit of the labels, 13 overthinking and 7 overacting states remain, and the overthinking axis (AUROC 0.983) barely beats position and reasoning length (0.949). Pushing the MLP edit all the way breaks the model rather than the decision: with the MLPs set exactly to Qwen's, 34/48 outputs are invalid (its only `FS+ answer` calls are on the infeasible context), and one pilot output repeats a single token over 91.6% of its 3,072 tokens.

These screens share the limits of §F.3: 16 samples per stratum come from two contexts, the circuit arms stopped after 5 outputs once the gate could no longer be met and never measured protected classes, and the MLP edit is an adaptation of the published recipe, not a numerical reproduction.

	tool calls				blind audit, fs+		identical to original	
Weight edit	fs+ `answer`	fs− `answer`	healthy valid	invalid	false claim	useful	tokens	reasoning
Original (re-served)	16/16	15/16	16/16	0/48	16/16	0/16	–	–
DeltaNet gates → Qwen (H2, 48 matrices)	16/16	15/16	16/16	0/48	16/16	0/16	2/51	21/51
MLPs halfway to Qwen (H3, 96 matrices)	16/16	15/16	16/16	0/48	16/16	0/16	0/51	1/51
`</think>` input row → newline (H5, $\lambda = 0.5$)	16/16	15/16	16/16	0/48	16/16	0/16	4/51	39/51
Full-attention temperature $\times 0.5$ (H6, 8 vectors)†	13/16	16/16	16/16	0/48	13/16	0/16	–	–
DeltaNet gates, $\lambda = 2$ (H2, 48 matrices)	16/16	15/16	16/16	0/48	16/16	0/16	–	–
Language layers 20% toward Qwen (H9, 401 tensors)	16/16	16/16	16/16	0/48	16/16	0/16	–	–
RoPE share of the query gain $\times 0.5$ (H10, 8 vectors)	16/16	15/16	16/16	0/48	16/16	0/16	–	–
RoPE share of the query gain $= 0$ (H10)†	8/16	13/16	14/16	3/48	8/16	0/16	–	–
MLPs = Qwen (H3, $\lambda = 1$)	3/16	3/16	8/16	34/48	–	–	–	–

Table 8: Mechanism-level weight edits change the tokens, not the outcome. vLLM 0.23 on separate hardware, same 6 contexts and 8 seeds per arm; the original is re-served on the same backend. The blind audit reads each false-completion output and counts claims of success and useful GUI actions, whatever tool is called. "Identical to original" counts the 48 GUI and 3 text outputs equal to the paired original, in full or up to the end of reasoning. One of the two FS+ contexts is an infeasible task, where `answer` explaining the impossibility is the correct action (Appendix Table 6). †Moving the tool is not a correction: every non-`answer` call falls on the infeasible context and is an unhelpful action justified by an incorrect prediction that it will restore the old interface (H6: 3 theme clicks; H10, $s = 0$: 7 theme clicks and 1 wait), none admitting the impossibility; none is useful. H10 at $s = 0$ is also rejected for truncated outputs on healthy steps, a loss of at least 12.5 points against a 10-point limit. Development pilot.

O Compiling an intervention into the weights

On the 4B no steering candidate passed the gate, so none was compiled; the weight edits of §F.5 were built directly by their own constructions. On the 0.8B, edits of the form below, restricted to top-k columns and with α from 0.3 to 16, were built and screened directly (§H). We state the compilation step because it constrains which steering interventions are worth testing. Let the residual stream at layer ℓ be $x_\ell = e + \sum_{j<\ell}(a_j + m_j)$, where $a_j = W_j^O z_j^a$ and $m_j = W_j^{\text{down}} z_j^m$ are the attention and MLP writes and e the token or image embedding, and let $P = I - uu^\top$ for a unit vector u ($\|u\| = 1$).

Proposition 1 (Directional ablation equals writer orthogonalisation; cf. [2])**.** *In a pre-norm decoder whose sublayers read the residual stream only through their normalisation, and whose residual writers have no bias except the output projection of the vision merger (W^{mrg}, bias b^{mrg}), replacing every writer $W \in \{W_j^O, W_j^{down}, W^{mrg}\}$ by PW, the embedding by $W_E P$ and the merger bias by Pb^{mrg} gives, in exact arithmetic, the same residual stream at every layer and position as the original model with $x \leftarrow Px$ applied after every write. The logits are identical too if the output head is kept unedited and untied; a tied head is projected with W_E, and since the final RMSNorm gain does not commute with P, the logits then differ.*

Proof sketch. By induction over writes: if x has no u-component, every normalised sublayer input is unchanged, so every z is unchanged, and each new write $PWz = P(Wz)$ keeps the sum orthogonal to u. □

Four consequences follow. A hook at block outputs is not equivalent, because the MLP then reads the un-ablated attention output. $W \leftarrow (I - \alpha uu^\top)W$ with $\alpha \neq 1$ scales each write's component by $1-\alpha$, whereas a residual hook compounds over layers. Tied embeddings (tied in both models) mean that projecting W_E also projects the output head unless it is untied. And an addition $h \leftarrow h + \beta u$, like every intervention in Table 2 except projections, can be compiled only as a bias, which the bias-free projections of Holo-3.1 do not have. The circuit arm is the exception: scaling heads and neurons is exactly a column scaling of W^O and W^{down}, 1,991,680 weights or 0.0439% of the model. It was tested in generation only as the equivalent hook, with no effect; no edited checkpoint was built. Any compiled patch must reproduce the steered behaviour within 5 points per stratum before serving. The serving path is ready: a patched checkpoint is loaded by vLLM in about seven minutes, a null patch changes 0/724 tensors and reproduces the original outputs. Since no 4B candidate passed, no compiled edit or equivalence check was run on the 4B.

P OSWorld diagnostic by domain

Table 9 breaks down the categories of §C by OSWorld domain.

Domain	Tasks	Success	Partial	FS	Step lim.	Loop	Other + unsc.
multi-apps	31	13/30	1	6	9	0	2
chrome	15	9/15	0	4	1	1	0
lo-calc	15	8/15	0	5	1	1	0
lo-impress	15	10/15	1	2	1	1	0
gimp	9	4/9	0	4	1	0	0
lo-writer	8	5/8	0	1	1	1	0
os	8	4/8	0	2	2	0	0
vs-code	8	6/8	0	1	0	1	0
vlc	6	2/6	0	4	0	0	0
thunderbird	5	5/5	0	0	0	0	0
Total	120	66/119	2	29	16	5	2

Table 9: Diagnostic split by domain (Holo-3.1-4B, pass 1). Success counts tasks scored 1, including correct refusals of infeasible tasks; Partial: tasks scored strictly between 0 and 1; FS: false success; Other + unsc.: unclassified failures plus unscored tasks; lo: LibreOffice.

Q Value-axis construction

The value direction contrasts success with failure steps of the same relative stage, averaged within task, on 68 construction tasks; the commitment direction contrasts `answer` with continuation steps. Neither uses the FS+/FS− labels. Evaluated on FS+ versus FS− under task-grouped cross-validation, the value direction scores 0.15 to 0.20 at layers 18, 24 and 28: it is higher on true completions, and read with the opposite sign it separates the two classes at 0.80 to 0.85, on par with step index and trajectory length (about 0.85), not beyond them. The separation it has is the one position already gives, the same confound as Trap 1.

R Holo-3.1-0.8B on MiniWoB++: details

The tables below give the 0.8B study of §H in full: the diagnostic (Table 10), the offline screening of weight edits (Tables 12 and 15), the steering arms (Table 13) and the per-task results (Table 14).

Primary category	Rollouts	% of failures
False success	37	36.3
Loop	25	24.5
Grounding miss	21	20.6
Abandon / budget	10	9.8
Wrong final action	9	8.8
Total failures	102	100.0

Table 10: Primary failure category, 0.8B (102 failed rollouts).

Layer	Contexts	min cosine	max rel. error
0	40	0.99995	0.0104
10	40	0.99951	0.0314
23	40	0.99694	0.0787

Table 11: Replay fidelity on MiniWoB++: replayed vs. recorded residuals.

Split	Steps	n	Decrease	95% CI (task bootstrap)	Random dir.
Dev (seeds 0–5)	loop+	72	3.89	[1.32, 6.41]	–
	healthy	344	9.97	[6.62, 13.69]	–
	loop−	287	9.16	[6.51, 11.63]	–
	success	119	10.86	[3.99, 20.58]	–
Reserved (seeds 6–7)	loop+	23	2.31	[-0.67, 7.62]	1.30
	healthy	119	8.92	[5.07, 14.03]	4.19
	loop−	90	10.33	[6.73, 14.81]	4.20
	success	54	9.42	[2.68, 19.39]	5.33

Table 12: Frozen 0.8B weight edit vs. original: mean decrease in log-probability per token of the recorded action (mnat/token), task-level bootstrap CIs. The random-direction column uses the same layers, writers, α and top-k.

Arm	REPEAT % [95% CI]	HEALTH % [95% CI]	Gens	Status
Original (no hook)	95.8 [88, 100]	75.0 [58, 90]	96	complete
Original, re-run	95.8 [88, 100]	70.8 [52, 88]	96	complete
Hook, $\alpha = 0$	95.8 [88, 100]	72.9 [54, 90]	96	complete
Random dir., all layers	95.8 [88, 100]	72.9 [54, 90]	96	complete
u_{10} at L10	93.8 [85, 100]	66.7 [48, 85]	96	complete
u_{10} at L9–13	89.6 [79, 100]	60.4 [40, 79]	96	complete
u_{10}, all layers	93.8 [83, 100]	64.6 [42, 85]	96	complete
u_{10}, all layers, $\alpha = 2$	93.8 [83, 100]	56.2 [35, 75]	96	complete
per-layer u_ℓ, all layers	81.2 [65, 94]	58.3 [40, 77]	96	complete
matched-pair dir.	n/m	≤ 66.7	36	rejected early
matched-pair PCA	n/m	≤ 62.5	36	rejected early
MLP-site dir.	n/m	≤ 66.7	36	rejected early
MLP-site PCA	n/m	≤ 62.5	40	rejected early
mean of layer dirs.	n/m	≤ 64.6	40	rejected early
strict pairs	n/m	≤ 62.5	36	rejected early
strict pairs PCA	n/m	≤ 64.6	40	rejected early
strict MLP	n/m	≤ 68.8	36	rejected early
strict MLP PCA	n/m	≤ 66.7	36	rejected early
u_{10} amplification $\alpha = -1$	n/m	n/m	4	aborted (invalid)
constant addition $-0.25\, u_{10}$	n/m	n/m	4	aborted (invalid)

Table 13: 0.8B free-generation steering arms (12 loop + 12 healthy contexts, $k = 4$). "Rejected early" arms show an upper bound on HEALTH that already fails the threshold.

Task	Success	Task	Success
`click-option`	7/7	`click-menu`	2/7
`click-tab`	8/8	`use-autocomplete`	2/7
`email-inbox-delete`	7/7	`click-tab-2`	1/7
`click-checkboxes`	7/8	`form-sequence`	1/7
`click-widget`	7/8	`login-user`	1/7
`navigate-tree`	6/7	`login-user-popup`	1/7
`click-dialog-2`	6/8	`multi-layouts`	1/7
`focus-text-2`	3/7	`social-media`	1/7
`enter-text-2`	3/8	`click-collapsible`	0/7
`click-checkboxes-soft`	2/6	`enter-date`	0/7
`choose-list`	2/7	`enter-password`	0/7
`click-button-sequence`	2/7	`search-engine`	0/7

Table 14: 0.8B success per MiniWoB++ task.

Candidate	Layers	α	top-k	Writers	Weights	$D_{\text{loop+}}$	D_{healthy}	$D_{\text{loop}-}$	D_{succ}	Pass
r3_distributed_a16	9–23	16.0	64	mlp+attn	1,966,080	262.66	286.96	259.44	342.09	no
r3_early_a16	9–10	16.0	256	mlp+attn	1,048,576	206.20	228.26	197.14	284.70	no
r3_L10_a16	10	16.0	64	mlp+attn	131,072	27.34	34.61	31.67	43.46	no
r3_distributed_a4	9–23	4.0	64	mlp+attn	1,966,080	13.96	6.69	8.61	3.29	no
a2_r3_a3	9–23	3.0	64	mlp+attn	1,966,080	9.31	-1.07	2.23	-6.32	yes
r3_early_a4	9–10	4.0	256	mlp+attn	1,048,576	8.11	14.12	8.68	17.66	no
a2_r3_a25	9–23	2.5	64	mlp+attn	1,966,080	7.08	-2.74	0.18	-7.62	yes
r3_mit_layers_a16	15,19	16.0	64	mlp	131,072	6.53	1.14	-1.04	2.85	no
a2_r2_distributed_a2	9–23	2.0	64	mlp+attn	1,966,080	4.72	-3.71	-1.26	-8.14	yes
r2_distributed_None	9–23	1.0	all	mlp+attn	86,507,520	4.47	1.05	1.89	0.25	no
a2_r3_mlp	9–23	2.0	64	mlp	983,040	3.40	-2.03	0.55	-5.25	yes
r3_L10_a4	10	4.0	64	mlp+attn	131,072	3.28	1.43	0.66	1.70	no
r2_early_None	9–10	1.0	all	mlp+attn	11,534,336	2.77	0.13	0.32	-1.45	yes
a2_r3_k32	9–23	2.0	32	mlp+attn	983,040	2.49	-3.59	-2.42	-6.75	yes
r1_L10_mlp_attn_None	10	1.0	all	mlp+attn	5,767,168	2.11	-0.13	0.73	-1.22	no
a2_r3_attn	9–23	2.0	64	attn	983,040	2.07	-2.85	-2.16	-4.49	yes
r2_distributed_64	9–23	1.0	64	mlp+attn	1,966,080	1.99	-5.19	-3.24	-8.60	yes
r2_late_attn_None	10,13,15,17,18,19,20	1.0	all	attn	14,680,064	1.89	-3.80	-2.32	-6.17	yes
a2_r3_k16	9–23	2.0	16	mlp+attn	491,520	1.88	-3.50	-2.93	-5.92	yes
r1_L10_attn_None	10	1.0	all	attn	2,097,152	1.77	-1.08	-0.38	-2.30	yes
r3_mit_layers_a4	15,19	4.0	64	mlp	131,072	1.38	-1.64	-1.08	-1.82	yes
a2_r2_distributed128	9–23	0.7	128	mlp+attn	3,932,160	1.38	-3.61	-3.23	-5.62	yes
r2_late_attn_64	10,13,15,17,18,19,20	1.0	64	attn	458,752	1.37	-3.25	-2.34	-5.11	yes
a2_r1_matched_mlp_attn	10	0.5	64	attn	65,536	1.13	-0.30	-0.52	-0.53	yes
a2_r1_matched_resid_attn	10	0.5	64	attn	65,536	1.02	-0.00	0.33	-0.45	yes
a2_r2_matched_early	9–10	0.7	128	attn	262,144	0.84	-0.12	-0.89	-0.23	yes
r1_L10_mlp_attn_64	10	1.0	64	mlp+attn	131,072	0.84	-0.43	0.04	-1.27	yes
r1_L10_attn_64	10	1.0	64	attn	65,536	0.83	-0.78	-0.67	-1.57	yes
a2_r2_matched_late	10,13,15,17,18,19,20	0.7	64	attn	458,752	0.76	-0.01	0.13	-0.45	yes
a2_r1_mlp_attn	10	0.5	64	attn	65,536	0.68	0.99	0.71	1.08	no
a2_r2_matched_attn16	10	0.3	16	attn	16,384	0.63	-0.44	-0.29	-1.30	yes
r2_mit_layers_None	15,19	1.0	all	mlp	7,340,032	0.63	-0.14	0.53	-0.11	no
r1_L10_mlp_64	10	1.0	64	mlp	65,536	0.53	-0.65	-0.53	-1.14	yes
r2_mit_layers_64	15,19	1.0	64	mlp	131,072	0.53	-0.30	0.05	-0.54	yes
a2_r1_matched_resid_mlp	10	0.5	64	mlp	65,536	0.52	-0.41	0.21	-1.35	no
a2_r2_matched_attn128	10	0.7	128	attn	131,072	0.52	0.03	-0.49	-0.19	yes
a2_r1_mlp_mlp	10	0.5	64	mlp	65,536	0.43	-0.80	-0.64	-1.46	yes
r1_L10_mlp_None	10	1.0	all	mlp	3,670,016	0.34	-0.21	0.14	-0.71	no
r2_early_64	9–10	1.0	64	mlp+attn	262,144	0.30	-1.33	-0.60	-2.79	yes
a2_r1_matched_mlp_mlp	10	0.5	64	mlp	65,536	0.18	-0.72	-0.03	-1.32	yes

Table 15: All offline weight-edit candidates on the 0.8B (40 fixed development contexts). D: decrease in mean log-probability per token of the recorded action (mnat/token). "Pass": $D_{\text{loop+}} > 0$ and $D_{\text{loop+}} \geq 3\max(0, D_{\text{healthy}}, D_{\text{loop}-}, D_{\text{succ}})$.

S How the research was run

The work ran from 7 November 2026, 14:37 UTC, to about 12:45 UTC on 8 November. This appendix condenses the working traces: `docs/lab_notebook/LOG.md`, `docs/lab_notebook/JOURNAL.md` (both mostly in French), `docs/RESEARCH_SYSTEM.md`, the protocol files of each folder and the git history.

Organisation. One Claude Code session (Opus) was the orchestrator. It set the questions and the GPU priority, froze the task splits, gave orders to the workers and owned every online run. The workers were Claude Code sessions and Codex CLI sessions running GPT-6 Astra, on one server, linked on a supervision canvas through which a session can read or write another's terminal. The orchestrator's log counts about 25 agents; this is a lower bound, and neither the exact number of sessions nor the peak number running in parallel was recorded. The traces name 26 sessions on weight edits and their diagnosis, and others for the evaluation infrastructure, the verifier studies, the paper and the demo. Two rounds of fresh-eyes reviews (adversarial, creative, outsider, physicist, philosopher) ran during the night.

Human team. Martin Bonan, Émile Jouannet and Souad Boukhedenna gave instructions from one shared terminal. The agents could not tell who was speaking and the traces write "Martin" or "the user" for all of them, so they cannot attribute a decision to a person. The team kept the budget, the approvals and the irreversible actions. Decisions recorded as the team's include the move from MiniWoB to OSWorld on the 4B model, the goal of "+10 proven points on OSWorld from weight edits alone" (not reached), the instruction at 01:27 on 8 November to continue after the first negative closures, the cost-benefit amendment of the online guard registration, and the extra Modal budget after the GPU pod was lost.

Worker rule. One track was one agent with a distinct hypothesis. Each worker owned one folder and one question, wrote its budget and stop rule before starting, and treated other folders as read-only. Budgets were counted in batches (failures included), forward passes or a deadline, never in dollars per worker; the dollar cap was per Modal account ($25, alert at $20, stop at $24). The default stop rule was two rounds in a row without measurable progress.

Safeguards. Splits, labels, the admission gate and the decision rules were committed before the data they judge; the repository holds 32 protocol, registration or amendment files. A gain is claimed only if the lower bound of the paired 95% CI is above zero. Results were checked by fresh-context agents that had not written the code. Two adversarial reviews on 8 November changed the paper: they found that 9 of the 29 false successes are infeasible tasks, that the logit margin was read after the agent had written "task completed", and that the original and the edits were served by different servers. The 45-task reserve and the TEST split were never used to build an edit, no weight or prompt was changed to make a check pass, and corrections were appended without rewriting the original entry. Errors caught before they reached the results include early "4B" runs that actually loaded the 0.8B model, and a guard rule that the code did not apply, fixed before the verifier was frozen at `0af76fdb`.

Rules not respected. The traces record five. (i) The note-to-tool attribution track failed twice at the same gate and then tried one more repair, a deviation from its two-round stop rule that its protocol states; its budget had also been raised from 6500 to 8500 forward passes. (ii) The guard's repair measure was written before its branches were launched but first committed when about six had finished, so its registration is self-declared. (iii) The TEST split was read before the analysis ended: ExPO was run on it without passing the gate. (iv) The decision rule of the weight-edit test was fixed before the Modal-served comparisons but after the first runs served by the pod had been seen. (v) For the stronger guard judge, the frozen verifier's held-out score was printed before any new run, and the variant was chosen after its development results, so its single held-out run is a confirmation, not a registered test.

Blockers. The rented H200 pod was at first slow (25 to 50 s per step) because of a 10-CPU quota, and had no KVM, so the OSWorld virtual machines ran in Docker with KVM on the team's own server. A vLLM server froze from 00:44 to 03:22 on 8 November, which invalidated one arm and

led to a watchdog for frozen engines; the pod disk also filled up several times. Around 09:30 on 8 November the provider deleted the pod on a low balance. The online guard registration was amended at 09:40 to use the original served by Modal as comparator, the team authorised $28 on one Modal account, and six tasks lost to infrastructure were replayed under a third amendment.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and §1 claim a diagnosis, three measured traps, a causal record in which no intervention passes the gate selectively, a mechanistic account of that failure and an offline and online evaluation of an external verifier; §2–§6 and the appendices support each claim, and the abstract states that the screens are pilots that detect only a fall to 4/8 false answers on a single feasible false-success context, and that in the verifier's single pre-registered TEST run the false "done" rate fell with no measurable score change, but most of the drop came from episodes in which the verifier never ran, so the drop is not attributed to it (no score gain is claimed).

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §6 (Limitations) and Appendix K: small causal samples that exclude only large effects, infeasible screening contexts, a three-context ceiling test, an exploratory direction analysis, reuse of the reserve and TEST split by non-admitted arms, many uncorrected comparisons, self-registered rules (the primary online rule was fixed after the merge's reserve scores were read), automatic labels and LLM auditors not validated against humans, a third-party harness, possible donor and verifier contamination, a single online guard run with a proprietary verifier, and a single model and benchmark.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The paper's one formal statement, Proposition 1 (directional ablation equals writer orthogonalisation), states its assumptions (a pre-norm decoder whose sublayers read the residual stream only through their normalisation, bias-free writers, every writer and the embedding projected); a proof by induction over writes follows it in Appendix O; it is short and given as a sketch, which we flag here.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Models and revisions, harness commit, sampling settings, frozen split hashes, the gate, seeds and every intervention's definition are given in §2–§4 and Appendices B–L; every number is regenerated by script from released result files.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code, frozen splits, raw generations, audits and the scripts that rebuild every number, table and figure will be provided to reviewers as an anonymised supplementary archive; raw outputs are released upon publication. Models and benchmarks are public, except the Claude verifier and judges, a proprietary service whose prompts we give and whose exact served version was not recorded.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sampling parameters, contexts, seeds, doses, layers and the selection procedure of every arm are in §3, §4, Table 2 and Appendix F. No model is trained.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Success rates and recurrence rates carry Wilson 95% intervals, each fs+ answer rate in Table 2 has its Wilson interval, and 0.8B results use task-level bootstrap intervals. We state that the intervals ignore clustering by task and that the screens can only detect large effects.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We report the hardware (one NVIDIA H200 shared with the rollout server, rented H100 GPUs for serving edited checkpoints, a separate CPU host for the VMs; Appendix B) and the budget of forward passes and generation batches of each arm, with order-of-magnitude estimates of total compute (Appendix B): about 14 hours of one H200, shared by all research jobs and the rollout server, so per-experiment GPU-hours are not separable; about 8 H100 GPU-hours; about 60 CPU jobs for the OSWorld VMs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses public models and benchmarks, involves no human subjects or personal data, and reports negative results and limitations in full.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: §6 (Broader impact): false success claims mislead users and downstream automation; our results argue for external verification and caution with interpretability-based repairs. The recommended fix sends desktop screenshots to a third-party model, a privacy cost, and about one refusal in four would hit a true success (§G); we release no edited model.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: We release no model or scraped data; edited checkpoints were used for evaluation only.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All assets are cited, with licences in Appendix B: Holo-3.1, Qwen3.5-4B, the CUA-Lite donor, OSWorld and BrowserGym are Apache-2.0 and MiniWoB++ is MIT; the RealtimeGUI harness has no licence file at the cited commit, which we state, and we use it unmodified except for the model provider and three VM-provider infrastructure settings (Appendix B). Holo4-27B, Fara1.5-4B and TMax-4B are used as published by their authors for evaluation only, and the Claude verifier and judges through Anthropic's service under its terms of use.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The released code and result files are documented: each generated macro names its source file, and the repository describes the pipeline, the protocol and the budget ledgers.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Experiments were planned, run and audited by LLM agents (Claude Code sessions and Codex agents; Appendix L) under human supervision, some protocol decisions were taken by an LLM coordinator, and LLMs are method components (the Claude verifier and judges, and the semantic audits of changed actions), following the protocol of §3; every result was audited by an agent that did not produce it (Appendix L). LLMs also assisted in writing.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.